

АННОТАЦИЯ

В выпускной квалификационной работе исследуется применение алгоритма адаптивного квантования для улучшения качества маломощных синусоидальных сигналов, полученных в процессе цифро-аналогового преобразования в генераторах сигналов.

Работа состоит из введения, шести глав, заключения, списка использованных источников и четырех приложений. Основной текст иллюстрирован 27 рисунками и содержит 4 таблицы. Список литературы включает 16 наименований.

В первой главе рассматриваются основные теоретические аспекты для понимания процесса исполнения алгоритма адаптивного квантования.

Во второй главе обосновывается выбор компонентов схемы, приведено соединение элементов схемы.

В третьей главе представлен принцип работы каждого блока программы, выполняющей алгоритм.

Четвёртая глава посвящена экономической части работы.

В пятой главе описаны охрана труда, режим труда и отдыха, общие требования к организации рабочего пространства.

В шестой главе приведены варианты импортзамещения микроконтроллерной платы.

ОГЛАВЛЕНИЕ

АННОТАЦИЯ	1
ВВЕДЕНИЕ	7
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АДАПТИВНОГО КВАНТОВАНИЯ В ИЗМЕРИТЕЛЬНЫХ ГЕНЕРАТОРАХ СИГНАЛОВ	10
1.1 Основные понятия и определения	10
1.1.1 Спектр сигнала и гармоники	10
1.1.2 Частота дискретизации и отсчёты	10
1.1.3 Алиасинг	11
1.1.4 Разрядность и уровни квантования	11
1.1.5 Ступенчатость сигнала	12
1.1.6 Сигнал/шум квантования	13
1.2 Практическая значимость исследования	14
1.2.1 Важность высокой точности поверки счётчиков электроэнергии	14
1.2.2 Проблема обеспечения высокой точности при малых сигналах в генераторах фиктивной мощности (ГФМ)	14
1.2.3 Подтверждение проблемы в научных статьях	15
1.3 Решение проблемы – алгоритм адаптивного квантования	15
1.3.1 Роль ЦАП в алгоритме	15
1.3.2 Критерий оптимальности	16
1.3.3 Преимущества получаемые при использовании алгоритма адаптивного квантования	16
1.4 Алгоритм адаптивного квантования	17
1.4.1 Способы округления сигнала без применения адаптивного квантования	17
1.4.2 Шаги при выполнении алгоритма	18
1.4.3 Моделирование идеального N-разрядного ЦАП	18
1.5 Принцип работы интерфейса обмена данными	19
1.5.1 Общие сведения об интерфейсе	19
1.5.3 Схема подключения устройств	20
1.6 Формирование сигналов на логическом и физическом уровне	21
1.6.1 Формирование START и STOP бит	21
1.6.2 Передача байта данных	21
1.6.3 Передача байта подтверждения	21
1.6.4 Формат адресного байта	22
2 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ АЛГОРИТМА АДАПТИВНОГО КВАНТОВАНИЯ НА ЯЗЫКЕ ПРОГРАММИРОВАНИЯ С С ИСПОЛЬЗОВАНИЕМ МИКРОКОНТРОЛЛЕРНОЙ ПЛАТЫ ARDUINO UNO	23
2.1 Выбор компонентов и приборов	23
2.1.1 Микроконтроллерная плата Arduino	23
2.1.2 ЦАП МСР4725	24
2.1.3 Обоснование выбора интерфейса I2C	25
2.1.4 Обоснование выбора осциллографа DSO138	25
2.1.5 Подключение Arduino Uno и ЦАП через I2C, осциллографа DSO138	26
2.1.6 Согласование компонентов по напряжению и частоте	28
3 РЕАЛИЗАЦИЯ ПРОГРАММЫ, ВЫПОЛНЯЮЩЕЙ АЛГОРИТМ АДАПТИВНОГО КВАНТОВАНИЯ НА ЯЗЫКЕ ПРОГРАММИРОВАНИЯ С	30
3.1 Выбор языка программирования и среды разработки	30
3.2 Структура программы	30
3.2.1 Подключение библиотек	30
3.2.2 Объявление объектов констант, переменных	30

3.2.3 Вспомогательные функции для работы с числами, функции process_signal и calculate_and_print_error()	31
3.2.4 Блок формирования идеальной синусоиды.....	32
3.2.5 Блок квантования.....	33
Блок для получения результатов сигнала с округлением до ближайшего значения в каждой точке и исходного.	33
3.2.6 Блок, отвечающий за адаптивный цикл в программе.	34
3.2.8 Функция Setar и Loop.....	34
3.2.9 Программа для расчёта на персональном компьютере.....	36
4 ЭКОНОМИЧЕСКАЯ ЧАСТЬ	43
5 ОХРАНА ТРУДА	44
5.1 Режим труда и отдыха	44
5.2 Общие требования к организации рабочего пространства	44
6 ИМПОРТОЗАМЕЩЕНИЕ	45
ЗАКЛЮЧЕНИЕ	46
СПИСОК ЛИТЕРАТУРЫ	47
ПРИЛОЖЕНИЕ А	49
ПРИЛОЖЕНИЕ Б	54
ПРИЛОЖЕНИЕ В	61
ПРИЛОЖЕНИЕ Г	66

ВВЕДЕНИЕ

Актуальность

В настоящее время существует большое количество счётчиков электроэнергии, которые нужно поверять. Для поверки приборов учёта электроэнергии используются генераторы сигналов с цифровым управлением. Они должны работать в широком диапазоне с изменением значения по току – до 1000 раз и более. [1] Применение округления отсчётов до ближайшего значения, всегда до верхнего или всегда до нижнего значения сигнала приводит к увеличению погрешности в сравнении с применением алгоритма адаптивного квантования. В связи с этим реализация данного алгоритма является актуальной задачей, позволяющей снизить шум квантования без повышения стоимости схемы из-за использования дополнительных компонентов.

Проблема исследования

Существует проблема: цифро-аналоговые преобразователи (ЦАП), преобразующие числовые значения в напряжение имеют ограниченный набор уровней. При большом сигнале – используются все разряды ЦАП. Сигнал на выходе состоит из большого количества ступенек, следовательно, сигнал плавный, шум квантования низкий. При малом сигнале задействуются только младшие разряды ЦАП. Старшие разряды остаются не использованы. Сигнал на выходе в таком случае будет состоять из небольшого количества ступенек, следовательно, сигнал будет иметь более высокий шум квантования, низкий динамический диапазон, вместо плавной синусоиды на выходе можно получить сигнал похожий на меандр.

Нередко эту проблему решают использованием более дорогого высокоразрядного ЦАП, например, 16, 18, 24 бита. Тогда и при малом сигнале будет много уровней напряжения. Или же используют программируемые усилители, где на вход ЦАП подаётся сигнал с высокой амплитудой за счёт умножения значения сигнала на коэффициент усиления для задействования всех разрядов ЦАП. На выходе снижается коэффициент усиления и сигнал становится низким.

Оба способа делают схему дороже и сложнее в реализации. В связи с этим задача упростить схему и сделать её дешевле актуальна. Алгоритм адаптивного квантования способен решить эту проблему и снизить шум квантования в ЦАП низкой разрядности без повышения стоимости и добавления новых элементов в схему. [5]

В нынешнее время существует такая проблема для систем цифро-аналогового преобразования (ЦАП), как эффективное использования разрешения при широком диапазоне сигнала на выходе. Шумы квантования вырастут при использовании ЦАП при низком уровне сигнала. [6]. Алгоритм адаптивного квантования способен решить данную проблему [4].

В работе рассматривается метод адаптивного квантования применительно к цифро–аналоговому преобразованию синусоидальных сигналов в генераторах сигналов.

Предложен подход, позволяющий динамически регулировать уровень выходного сигнала и минимизировать влияние шумов, возникающих, при работе с низким разрешением.

Объект исследования

Применение алгоритма адаптивного квантования в процессе цифро-аналогового преобразования синусоидальных сигналов в генераторах сигналов.

Предмет исследования

Алгоритм адаптивного квантования, позволяющий снизить шум квантования, возникающий в процессе цифро-аналогового преобразования.

Научную новизну в данной работе представляет собой разработка программы, реализующей выполнение алгоритма адаптивного квантования на микроконтроллерной плате с последовательным подбором уровней напряжения при цифро-аналоговом преобразовании. Написание программы осуществлялось с опорой на патент [13].

В работе реализован подход, при котором процесс оптимизации (адаптации) параметров сигнала происходит непосредственно на целевом устройстве, а не на этапе подготовки данных на ЭВМ. Практическая значимость состоит в создании действующего макета генератора сигналов с цифровым управлением, используя только микроконтроллерную плату и ЦАП, что позволит отказаться от программируемых усилителей и сложных выходных фильтров.

Целью исследования является реализация и экспериментальное подтверждение эффективности алгоритма адаптивного квантования в снижении погрешности первой гармоники и среднеквадратичного значения (СКЗ) синусоидального сигнала.

Задачи:

1. Изучение литературы по теме адаптивного квантования и измерительных генераторов сигналов.
2. Написание алгоритма адаптивного квантования на языке программирования C.
3. Реализация алгоритма адаптивного квантования на микроконтроллерной плате.
4. Проведение эксперимента с разным количеством отсчётов на период и разным значением амплитуды сигнала, где для каждого эксперимента рассчитывается амплитуда первой гармоники идеального сигнала, сигнала с округлением до ближайшего и после адаптации.

5. Сравнение разницы амплитуды первой гармоники при адаптивном квантовании и округлении до ближайшей каждой точки сигнала относительно идеального сигнала для дальнейшего выбора наиболее близкого к идеальному сигналу и его вывод на осциллограф.

6. Написание и выполнение программы на ПК, рассчитывающей погрешность первой гармоники и СКЗ при округлении до нижнего уровня, до верхнего, до ближайшего уровня и с применением алгоритма адаптивного квантования.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АДАПТИВНОГО КВАНТОВАНИЯ В ИЗМЕРИТЕЛЬНЫХ ГЕНЕРАТОРАХ СИГНАЛОВ

1.1 Основные понятия и определения

1.11 Спектр сигнала и гармоники

В данной работе под спектром сигнала понимается частотное представление исследуемого сигнала, выраженное в виде набора гармоник с различными амплитудами, фазами и частотами. Используя спектральный анализ, любой периодический сигнал может быть разложен на гармоники – сигналы с частотами кратными основной. При этом первая (основная) гармоника является носителем полезного сигнала. Высшие гармоники являются помехами, подлежащими минимизации и фильтрации.

1.1.2 Частота дискретизации и отсчёты

Одним из основных аспектов цифровой обработки информации является дискретизация – это процедура преобразования непрерывного аналогового сигнала в последовательность дискретных отсчётов (цифровых кодов).

Процесс дискретизации имеет два основных параметра:

Период – это промежуток времени, за который периодический сигнал проходит цикл от пика до впадины и возвращается в исходное состояние.

$$T = \frac{1}{f}, \quad (1.1)$$

Где T – это период; f – это частота.

Частота дискретизации – это величина, определяющая количество замеров (отсчётов) амплитуды сигнала в единицу времени.

Частота дискретизации ограничивается возможностями компонентов внутри ЦАП. Они не могут быстрее срабатывать или замерять информацию быстрее определённого предела.

$$f_d = \frac{1}{T_d}, \quad (1.2)$$

где T_d – период дискретизации (промежуток времени между двумя соседними замерами); f_d – частота дискретизации.

Ограничением для процесса дискретизации является теорема Котельникова-Найквиста. Согласно теореме для процесса дискретизации, частота дискретизации должна как минимум быть в два раза выше, чем частота преобразуемого сигнала. В случае невыполнения этого условия возникает явление алиасинга. [2]

$$f_s > 2 * f_{\max}, \quad (1.3)$$

где f_s – частота дискретизации; $f_{\max}$ – максимальная частота сигнала.

1.1.3 Алиасинг

Алиасинг – эффект, возникающий, в случае, когда частота обрабатываемого сигнала выше, чем частота Найквиста (половина частоты дискретизации). Явление характеризуется возникновением ложных частотных компонентов, искажающих выходной результат. Это происходит из-за слишком редких замеров. Происходит соединение точек так, что взамен высокочастотной волны создаётся её низкочастотный ложный сигнал.

$$f_{\text{алиас}} = |f_d * n - f_{\text{сиг}}|, \quad (1.4)$$

где $f_{\text{алиас}}$ – частота ложного сигнала; f_d – частота дискретизации; $f_{\text{сиг}}$ – частота исходного сигнала; n – целое число, представляющее собой номер частоты дискретизации, относительно которой возникает зеркальное отражение спектра (подбирается таким, чтобы результат являлся минимальным положительным числом) [2].

1.1.4 Разрядность и уровни квантования

Проблема алиасинга решается использованием фильтрации. После того как, пройдена фильтрация и устранены возможные помехи от алиасинга, то больше всего влияние на погрешность будет составлять шум квантования. Использование алгоритма адаптивного квантования снижает шум квантования.

При обычном подходе к округлению сигнала до ближайшего уровня возникает ошибка квантования (разница между реальным значением аналогового непрерывного сигнала и округленным цифровым значением в конкретной точке. ЦАП не может точно попасть в значение, например, в 1,4675234. Если ЦАП 2-битный, то у него 4 уровня. Это 0, 1,25, 2,5, 3,75 вольт. Данный ЦАП выдаст либо 1,25, либо 2,5 в зависимости от алгоритма, по которому происходит округление.

Шум квантования – разность между идеальным значением сигнала и квантованным.

Разрядность ЦАП определяет точность передачи мгновенных значений сигнала. Чем выше разрядность ЦАП, тем меньше ошибка квантования в конкретной точке и меньше шум квантования для всего сигнала. Квантование в ЦАП – это преобразование цифрового значения сигнала в реальную физическую аналоговую величину с фиксированным шагом.

1.1.5 Ступенчатость сигнала

Количество ступенек в сигнале, прошедшем через ЦАП, зависит от разрядности ЦАП (количества бит). ЦАП не может выдавать любое количество уровней, а только фиксированное. Также ЦАП не может выдать любое напряжение, а только кратное значению шагу квантования (самая маленькая ступенька, которую способен выдать ЦАП).

Например, при разрядности 8 бит у ЦАП: $2^8=256$ ступенек. Ступеньки у аналогового сигнала на выходе ЦАП будут крупные из-за чего при округлении возникнут заметные ошибки квантования.

Как в рассмотренном примере выше. При разрядности ЦАП 16 бит разрядность будет равной $2^{16}= 65536$ уровней напряжения. Для наглядности насколько значительно влияет каждый разряд у ЦАП на количество ступенек, ниже представлена Таблица 1. [15]

Таблица 1 – Степени числа, демонстрирующие насколько увеличивается количество уровней напряжения ЦАП при увеличении разрядности ЦАП.

2^0	1	2^8	256
2^1	2	2^9	512
2^2	4	2^{10}	1024
2^3	8	2^{11}	2048
2^4	16	2^{12}	4096
2^5	32	2^{13}	8192
2^6	64	2^{14}	16384
2^7	128	2^{15}	32768

При большом количестве ступенек переход от одной ступеньки к следующей небольшой, поэтому при более высоком значении разрядности ЦАП сигнал чище, шум квантования меньше.

1.1.6 Сигнал/шум квантования

Для представления эффективности алгоритма адаптивного квантования стоит упомянуть такие понятия и их влияние на сигнал, как отношение сигнал/шум и динамический диапазон.

Динамический диапазон – отношение максимального сигнала к минимальному, отличимому от шума.

$$D = \frac{U_{\text{макс}}}{U_{\text{мин}}}, \quad (1.6)$$

Где D – динамический диапазон; $U_{\text{макс}}$ – максимальное напряжение сигнала; $U_{\text{мин}}$ – минимальное напряжение сигнала, отличимое от шума.

При переводе отношения, считаемого в количестве раз в децибелы, то вместо деления считается вычитание. И получается разница от максимального сигнала до минимально отличимого от шума.

$$D = 20 * \log_{10} \left(\frac{U_{\text{макс}}}{U_{\text{мин}}} \right) \quad (1.7)$$

Отношение сигнал/шум (SNR) – это безразмерная величина, показывающая насколько уровень сигнала превышает уровень шума.

Для расчёта во сколько раз уровень сигнала превышает уровень шума используется формула:

$$\text{SNR} = \frac{U_{\text{сиг_ск}}}{U_{\text{шума_ск}}} \quad (1.8)$$

Где $U_{\text{сиг_ск}}$ – среднеквадратичное значение сигнала; $U_{\text{шума_ск}}$ – среднеквадратичное значение шума.

Для расчёта в децибелах насколько уровень сигнала превышает уровень шума используется формула:

$$\text{SNR} = 20 * \log_{10} \left(\frac{U_{\text{сиг_ск}}}{U_{\text{шума_ск}}} \right) \quad (1.9)$$

Существует проблема: при снижении уровня сигнала не задействуются старшие биты ЦАП. Например, в 12 битном ЦАП при сигнале с высокой амплитудой будет задействованы все биты. Поэтому в сигнале будет много мелких ступенек: $2^{12}=4096$. Если уменьшить сигнал в 100 раз, то старшие биты не используются. [6]

$$L = \frac{4096}{100} \sim 41 \text{ ступеней} \quad (1.10)$$

Где L – количество ступеней

$$N_{\text{исп}} = \text{Log}_2(41) \sim 5 \text{ бит} \quad (1.11)$$

1 бит разрешения соответствует приблизительно 6 децибелам динамического диапазона. Каждый бит увеличивает уровень громкости почти в 2 раза.

В данной работе качество сигнала будет оцениваться по двум критериям: по относительной погрешности первой гармоники и относительной погрешности СКЗ. Для каждого из четырёх методов округления (вверх, вниз, до ближайшего, адаптивное квантование).

1.2 Практическая значимость исследования

1.2.1 Важность высокой точности поверки счётчиков электроэнергии

Практическая значимость исследования подтверждается проблемой, описанной в статье в статье «Оценка погрешности дискретности при перемножении двух дискретных сигналов, [3] заданных в виде кодов». Существует потребность в обеспечении высокой точности в области поверки счётчиков электрической энергии.

1.2.2 Проблема обеспечения высокой точности при малых сигналах в генераторах фиктивной мощности (ГФМ)

В современных системах формирования эталонных сигналов, таких как ГФМ, существует проблема обеспечения высокой точности воспроизведения активной мощности в широком динамическом диапазоне.

Ограниченное число дискретных уровней выходного напряжения и дискретных отсчётов на период сигнала в ЦАП, используемых для формирования сигнала, приводит к возникновению методической погрешности.

Источником погрешности в данном случае является шум квантования, возникающий при преобразовании цифровых кодов в аналоговые сигналы посредством ЦАП.

При классическом подходе к цифро-аналоговому преобразованию, если уменьшить амплитуду сигнала, то старшие разряды ЦАП будут не задействованы. Что приведёт к росту шума квантования. Поэтому реализация алгоритма адаптивного квантования, позволяющего сохранить качество сигнала при работе ЦАП в широком динамическом диапазоне является актуальной задачей.

Например, активная мощность в цепи переменного тока вычисляется по формуле:

$$P=UI\cos\varphi \quad (1.12)$$

Где:

U – действующее значение напряжения

I – действующее значение тока

$\cos\varphi$ – косинус фазового сдвига между напряжением и током.

Любой периодический сигнал возможно разложить в ряд Фурье:

$$I(t)=I_1\sin(\omega t+\varphi_1)+ I_2\sin(2\omega t+\varphi_2)+ I_3\sin(3\omega t+\varphi_3)+\dots \quad (1.13)$$

Где:

I_1 – амплитуда первой гармоники;

I_2, I_3 – амплитуды высших гармоник.

Если первая гармоника имеет погрешность, то и мощность будет иметь погрешность. Снижение погрешности активной мощности в ГФМ – польза от алгоритма адаптивного квантования.

1.2.3 Подтверждение проблемы в научных статьях

В статье «Применение адаптивного квантования в измерительных генераторах с цифровым управлением» [4], предлагается метод адаптивного квантования для измерительных генераторов с цифровым управлением. Показано, что данный метод позволяет существенно снизить погрешность первой гармоники или подавить низкочастотные гармоники, а также уменьшить зависимость качества сигнала от частоты дискретизации.

1.3 Решение проблемы – алгоритм адаптивного квантования

1.3.1 Роль ЦАП в алгоритме

Функция ЦАП— это преобразование отсчётов формируемого сигнала, хранящихся в памяти генератора, в напряжение. Методическая погрешность дискретности — шум квантования, возникающий из-за квантования (дискретизации) по времени и по уровню отсчётов сигнала в ЦАП. [14]

От амплитуда формируемого сигнала будет зависеть относительное значение шума дискретизации. Старшие разряды ЦАП принимают участие в процессе преобразования, в случае если амплитуда сигнала высока. Что приводит к уменьшению относительного значения шума. При низкой амплитуде сигнала, шум квантования может стать недопустимо большим.

Адаптивное квантование может улучшить один из выбранных параметров сигнала (критерий оптимальности). Выбранным параметром сигнала может являться погрешность воспроизведения первой гармоники.

1.3.2 Критерий оптимальности

В работе используется критерий оптимальности для оценки эффективности предложенного алгоритма. Критерий оптимальности – математическое выражение, оценивающие насколько полученный результат соответствует цели исследования. [8]

В работе выбраны два критерия оптимальности:
относительная погрешность первой гармоники сигнала;
относительная погрешность СКЗ.

1.3.3 Преимущества получаемые при использовании алгоритма адаптивного квантования

Снижение зависимости выбранного параметра от частоты отсчётов и от количества используемых разрядов ЦАП также возможно при помощи адаптивного квантования. Это позволит без использования дополнительных усилителей с программируемым коэффициентом усиления расширить динамический диапазон генератора.

Поверочные установки с измерительными генераторами в составе (калибраторы фиктивной мощности) используются для поверки счётчиков электрической энергии.

Адаптивное квантование позволит уменьшить шум квантования при использовании малого количество разрядов ЦАП без применения программируемых усилителей.

Сигнал на выходе имеет вид ступенчатой функции с фиксированным числом уровней напряжения, меняющихся в определённые моменты времени. В отличие от непрерывного аналогового сигнала, результат работы ЦАП содержит ошибку квантования, зависящую от разрядности устройства, частоты отсчётов и от используемого алгоритма округления значений выходного сигнала.

1.4 Алгоритм адаптивного квантования

1.4.1 Способы округления сигнала без применения адаптивного квантования

Три способа округления отсчётов выходного сигнала возможны по уровню напряжения, они продемонстрированы на Рисунке 1:

До верхнего значения – 3, до нижнего значения – 4, до ближайшего значения – 2.

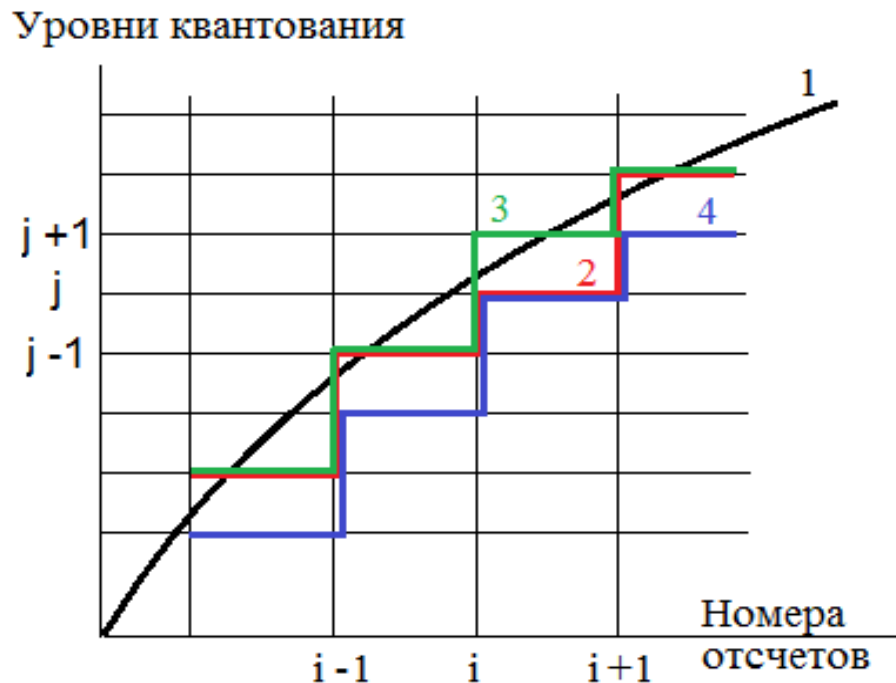


Рисунок 1 – способы округления

Среди представленным вариантов при использовании до ближайшего значения шум квантования наименьший. По модулю он не превысит половину уровня кванта. Округление до ближайшего значения будет использовано за основу при использовании алгоритма адаптивного квантования при реализации в программе на микроконтроллерной плате.

Если форма сигнала заранее неизвестна, то при цифро-аналоговом преобразовании сигнала ЦАП преобразует цифровые значения точек сигнала в соответствующие им уровни аналогового сигнала способом – округлением до ближайшего целого значения или постоянно вверх, или постоянно вниз.

В случае если формируемый сигнал известен (синусоидальный), возникает возможность оптимизации процесса квантования разными способами в реальном времени.

В работе под «адаптивным квантованием» подразумевается цифро-аналоговое преобразование с постоянным шагом по времени и по уровню кодов заранее известного сигнала.

Эти цифровые значения округляются так, чтобы адаптированный сигнал на выходе ЦАП был лучше по критерию оптимальности в сравнении с сигналом с другим способом округления.

В качестве критерия оптимальности может быть принят любой параметр сигнала. После чего проводится эксперимент для определения насколько алгоритм улучшит данный критерий.

1.4.2 Шаги при выполнении алгоритма

Этапы алгоритма адаптивного квантования

1. Получение идеального сигнала, с которым будет происходить сравнение.
2. Вычисление значения критерия оптимальности для эталонного сигнала и его запоминание..
3. Получение трёх вариантов округления.
4. Выбор наиболее подходящего значения по критерию оптимальности (с минимальной погрешностью).

В работе алгоритм реализован в двух версиях: на микроконтроллерной плате Arduino Uno (работа в реальном времени, сравнение округления до ближайшего и адаптивного квантования по первому критерию) и на ПК (сравнение четырёх методов по обоим критериям).

1.4.3 Моделирование идеального N-разрядного ЦАП

В научной статье «Применение адаптивного квантования в измерительных генераторах с цифровым управлением» [5] в результатах моделирования была создана модель идеального N-разрядного ЦАП. ЦАП формировал периодический дискретный сигнал с количеством M отсчётов на период, где шум квантования определялся только процессами дискретизации и квантования. Без зависимости от инструментальных погрешностей преобразователя.

В статье описан результат работы модели программы: при использовании округления до ближайшего уровня. Выявлено, что в этом случае погрешность дискретизации значительно зависит от частоты дискретизации: незначительные колебания частоты дискретизации или частоты генерируемого сигнала могут сильно увеличить погрешности. Для решения этой проблемы использован алгоритм адаптивного квантования.

Авторами статьи было получено, что алгоритм адаптивного в данном примере обеспечивает повышение точности воспроизведения первой гармоники более чем в 10 раз. При числе отсчётов на период более 20. При меньшем числе отсчётов повышение точности ниже.

Снижение погрешности основной гармоники было достигнуто за счёт перераспределения энергии сигнала в область высокочастотных составляющих.

В статье говорится, алгоритм также позволяет избирательно подавлять одну или несколько наиболее выраженных низкочастотных гармоник, что значительно упростит требования к выходной фильтрации.

В заключении статьи отмечено, что алгоритм адаптивного квантования может существенно сделать лучше разные выбранные параметры синусоидального сигнала, что формируется ЦАП в измерительных генераторах.

Алгоритм даёт возможность повышения точности воспроизведение первой гармоники синусоиды и возможность ухудшения второстепенных параметров для достижения выигрыша по выбранному критерию оптимальности.

1.5 Принцип работы интерфейса обмена данными

1.5.1 Общие сведения об интерфейсе

В выпускной квалификационной работе будет использован интерфейс I2C для передачи данных между устройствами. Подробная схема подключения, обоснование выбора для реализации алгоритма адаптивного квантования на практике будет представлено в практической части работы.

Без понимания принципов передачи данных по I2C возможны ошибки в тактировании, порядке байтов и адресации, что приведёт к искажению выходного сигнала. В связи с этим в разделе 1.5 рассматривается принцип работы I2C.

I2C (Inter-integrated circuit) – это последовательный интерфейс для связи цифровых модулей внутри устройства, разработанный Philips (сейчас NXP).

Главной особенностью интерфейса является подключение к двум линиям: SDA (линия связи), SCL (линия тактового сигнала – синхронизации).

К основным параметрам интерфейса относят:

- синхронную передачу данных
- наличие двух линий для передачи данных (ещё есть общая земля)
- скорость передачи данных до 400 кБ/с
- интерфейс полудуплексный
- помехозащищённость интерфейса ограничена

Помехозащищённость проявлена тем, что принимающее устройство подтверждает факт приёма байта, максимальная длина линии зависит от адресации. Обычно 127 устройств (для 7-битной адресации).

1.5.3 Схема подключения устройств

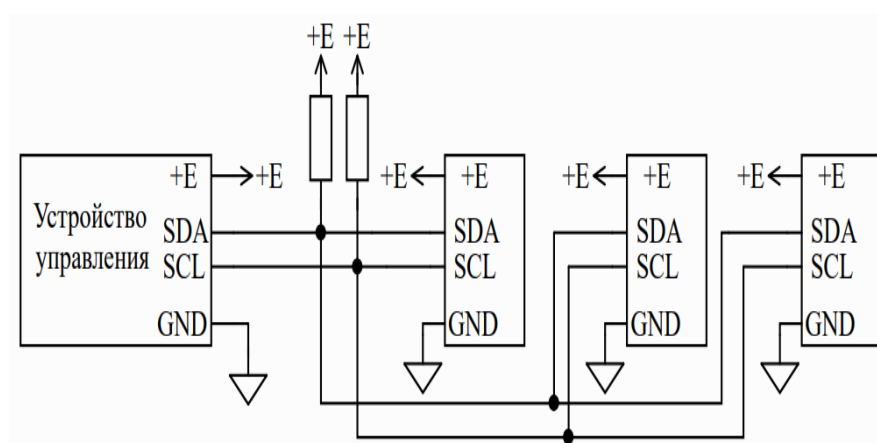


Рисунок 2 – Схема подключения устройств по интерфейсу I2C

I2C включает в себя две линии передачи данных. Первая – SDA (линия данных), вторая – SCL (линия синхронизации). К этим двум линиям подключаются все устройства сети. Обе линии подключены к питаниям через подтягивающие резисторы. Схема подключения устройств по интерфейсу I2C представлена на Рисунке 2.

1.6 Формирование сигналов на логическом и физическом уровне

1.6.1 Формирование START и STOP бит

I2C – синхронный интерфейс, поэтому передача бит данных происходит синхронно с импульсами, осуществляемыми ведущим устройством на линии SCL, притягивая линию к земле.

Напряжение близкое к напряжению питания должно находиться на линиях протокола перед началом процедуры передачи данных. Инициатором обмена данными выступает всегда ведущее устройство START бит характеризуется переходом линии SDA в состояние низкого уровня напряжения из высокого при высоком напряжении на линии SCL. STOP бит используется для завершения передачи данных. STOP бит формируется при переходе линии SDA из низкого логического уровня напряжения в высокий при сохранении высокого уровня напряжения на тактовой линии SCL.

1.6.2 Передача байта данных

Тактовые импульсы на линии SCL генерируются ведущим устройством. Осуществление передачи данных происходит таким способом: когда на линии SCL низкий уровень напряжения, то в процессе перемещения данных от одного устройства к другому по физическому каналу происходит изменение логического уровня SDA – передающее устройство подготавливает следующий бит к передаче.

Но принимающее устройство при линии SCL в состоянии равно нулю не будет считывать данные. При фиксации высокого уровня сигнала на SCL – возможно считывание данных принимающим узлом.

1.6.3 Передача байта подтверждения

Данные передаются по 8 бит, полный цикл обмена включает 9 тактовых импульсов. Девятый импульс предназначен для сигнала подтверждения ACK. Передатчик переводит линию SDA в высокоомное состояние, а приёмник притягивает её к земле, сигнализируя об успешном получении байта. Ситуация, когда отсутствует такое подтверждение отсутствует считается как NACK. Временная диаграмма передачи байта данных по интерфейсу I2C показана на Рисунке 3.

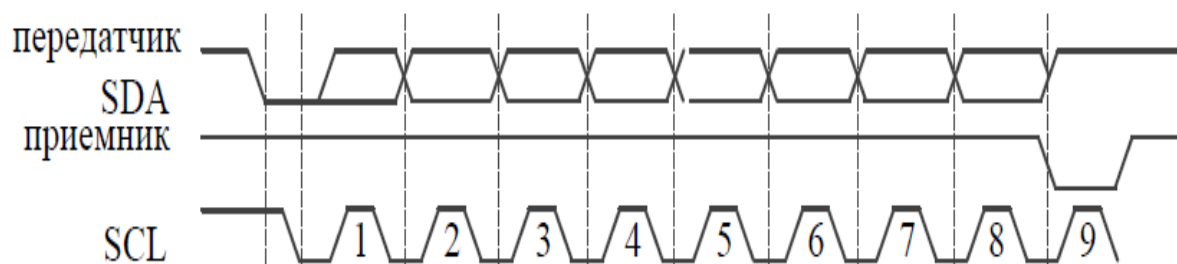


Рисунок 3 - Временная диаграмма передачи байта данных по интерфейсу I2C

1.6.4 Формат адресного байта

Передача начинает осуществляться с старшего разряда. Количество переданных байт определяется режимом передачи и типом устройства. [7]

Структура адресного сообщения по шине I2C к устройству – на рисунке 4.

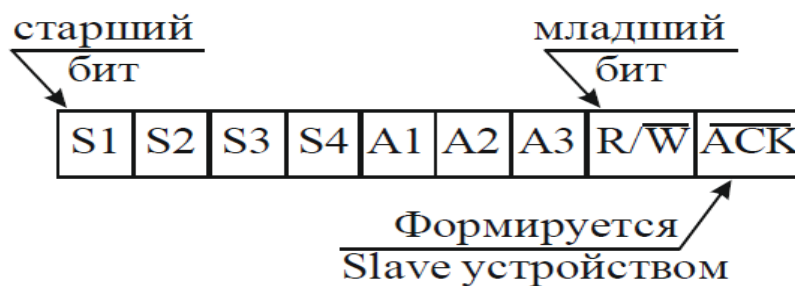


Рисунок 4 – Структура адресного сообщения по шине I2C к устройству

2 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ АЛГОРИТМА АДАПТИВНОГО КВАНТОВАНИЯ НА ЯЗЫКЕ ПРОГРАММИРОВАНИЯ C С ИСПОЛЬЗОВАНИЕМ МИКРОКОНТРОЛЛЕРНОЙ ПЛАТЫ ARDUINO UNO

2.1 Выбор компонентов и приборов

2.1.1 Микроконтроллерная плата Arduino

Решение использовать платформу Arduino Uno для практической реализации алгоритма адаптивного квантования было принято из-за нескольких факторов. Данная микроконтроллерная плата является распространённой, доступной и обладает невысокой стоимостью. Она была в наличии в лаборатории кафедры, что дало возможность начать проводить эксперимент по применению алгоритма адаптивного квантования без дополнительных затрат. Что значительно упростило практическую реализацию работы. Основой платы служит микроконтроллер ATmega328P. [10] Этот микроконтроллер поддерживает программирование на языке C, имея достаточную производительность для реализации алгоритма адаптивного квантования, позволяет реализовать эксперимент. Вывод данных через USB даёт возможность визуализировать полученные результаты на ПК. [9] Подробные технические характеристики микроконтроллерной платы представлены в приложении Б на Рисунок Б5 и на Рисунок Б6. Микроконтроллерная плата Arduino Uno представлена на Рисунке 5.



Рисунок 5 - Микроконтроллерная плата Arduino Uno

2.1.2 ЦАП МСР4725

ЦАП МСР4725 использован, потому что у микроконтроллерной платы Arduino Uno R3 нет аналогового выхода сигнала. Данный ЦАП имеет 12 разрядов, что даёт $2^{12}=4096$ уровней, что позволит экспериментально задействовать алгоритм адаптивного квантования в широком динамическом диапазоне. МСР4725 поддерживает интерфейс передачи данных I2C. Технические характеристики данного ЦАП приведены в приложении Б на Рисунок 1 и Рисунок 2. [16]

Изображение ЦАП МСР4725 продемонстрировано на Рисунке 6.

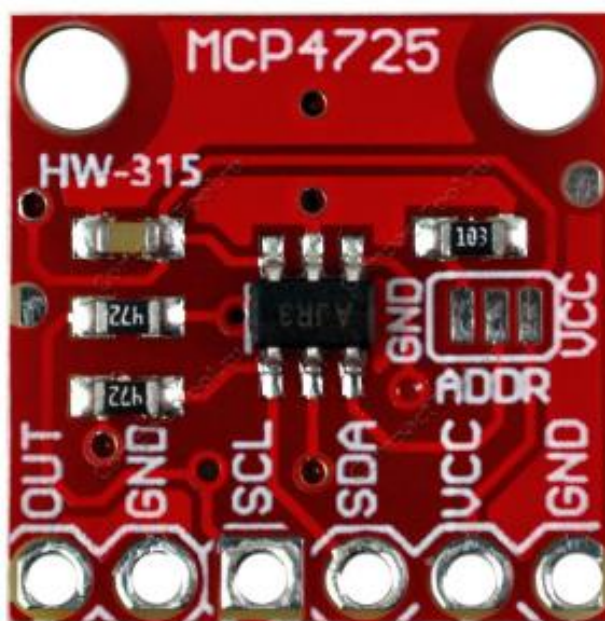


Рисунок 6 – ЦАП МСР4725

ЦАП необходим в схеме, потому что микроконтроллерная плата Arduino не выдаёт аналоговый сигнал. А только либо цифровой, либо ШИМ сигнал.

Данный модуль подключается по интерфейсу I2C в режиме Fast Mode на частоте передачи данных 400 кГц, обеспечивая скорость работы, позволяющую обрабатывать изменения сигнала в реальном времени.

Один сеанс обмена данными занимает 38 тактов. А именно: старт-бит, адресное слово (из 7 бит адреса, 1 бита записи, 1 бита подтверждения), байт команды (8 бит данных и бит подтверждения), байт данных из 8 бит и 1 бит подтверждения), байт данных 2 (из 8 бит и 1 бит подтверждения), стоп бит. Учитывая, что одна посылка занимает 38 тактов шины при частоте 400 кГц время передачи одного бита:

$$T_{\text{Передачи}} = \frac{38}{400000} = 95 \text{ мкс} \quad (2.1)$$

2.1.3 Обоснование выбора интерфейса I2C

Интерфейс I2C выбран для связи Arduino Uno с ЦАП MCP4725 из-за оптимального соотношения между скоростью передачи и использованием ресурсов микроконтроллерной платы. В I2C используются две линии передачи данных. Это позволяет использовать низкое количество выводов на Arduino, сохраняя остальные выводы свободными. Что даёт возможность использовать их для других целей. Например, для подключения датчиков.

Как было указано выше, интерфейс в режиме Fast Mode способен передавать данные на частоте до 400 кГц, а биты подтверждения приёма (ACK/NACK) обеспечивают контроль доставки данных между устройствами. В Arduino существует библиотека Wire.h, управляющая шиной на низком уровне (прямое управление логическими уровнями на физических линиях связи с соблюдением временных промежутков), что позволяет программисту сосредоточиться на самом алгоритме квантования сигнала. [10]

2.1.4 Обоснование выбора осциллографа DSO138

Осциллограф DSO138 выбран для визуальной проверки работы алгоритма адаптивного квантования. Выбранный осциллограф доступен и имеет достаточные характеристики для визуализации сигналов до 200 кГц. Также преимуществом данного осциллографа является его портативность, что позволяет проводить измерения сигнала дома. Что значительно превышает максимальный входной сигнал с частотой 83 Гц. Подробные технические характеристики осциллографа представлены в приложении Б в таблице Б1. [11] Изображение осциллографа продемонстрировано на Рисунке 7.

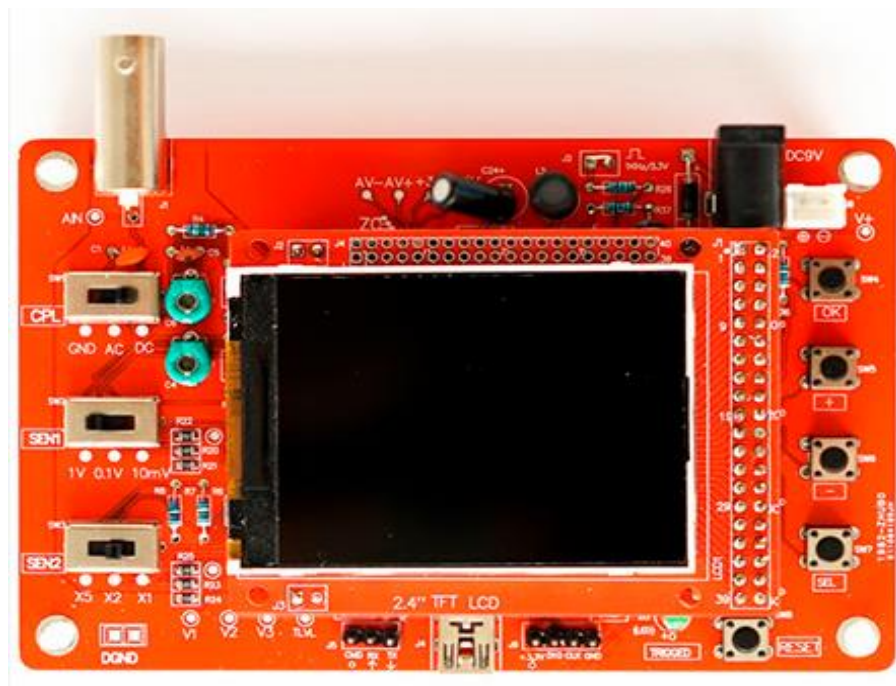


Рисунок 7 - Осциллограф DSO138

2.1.5 Подключение Arduino Uno и ЦАП через I2C, осциллографа DSO138

В схеме управляющим элементом микроконтроллерная плата Arduino Uno. Выводы платы могут давать только цифровой сигнал и ШИМ сигнал поэтому для получения аналогового непрерывного сигнала синусоиды в работе был использован внешний 12-битный ЦАП MCP4725. Подключение выводов данного ЦАП приведено в Приложении Б на Рисунок Б.3.

Устройства соединены между собой по интерфейсу передачи данных I2C. К преимуществам данного интерфейса относится использование малого количества выводов Arduino Uno, позволяя использовать выводы для других задач. Соединение представлено в Таблице 2.

Таблица 2 – Соединение контактов ЦАП, Arduino и осциллографа

Модуль MCP4725	Arduino Uno	Осциллограф DSO138	Примечание
1	2	3	4
VCC	5V	-	Питание
GND	GND	Крокодил	Земля
SDA	A4	-	Линия

			данных
1	2	3	4
SCL	A5	-	Линия частоты
OUT	-	Щуп	Выход аналогового сигнала

Взаимодействие Arduino Uno и модуля ЦАП в работе выполнено с использованием интерфейса передачи данных I2C. Вывод питания микроконтроллерной платы соединяется с выводом ЦАП VCC. Вывод GND Arduino соединён с выводом GND на ЦАП.

К проводнику сигнала GND между ЦАП и платой присоединён крокодил от осциллографа. Это обеспечивает общую землю в схеме. Линия данных SDA на ЦАП соединяется с аналоговым выводом A4 на Arduino, линия тактирования SCL соединяется с выводом A5. В качестве генератора сигнала выступает микроконтроллерная плата, а выходом генератора служит вывод OUT на ЦАП. На выходе Структурная схема приведена на Рисунок 8. На Рисунок 9 изображена фотография макета генератора сигнала.



Рисунок 8 – Структурная схема подключения компонентов

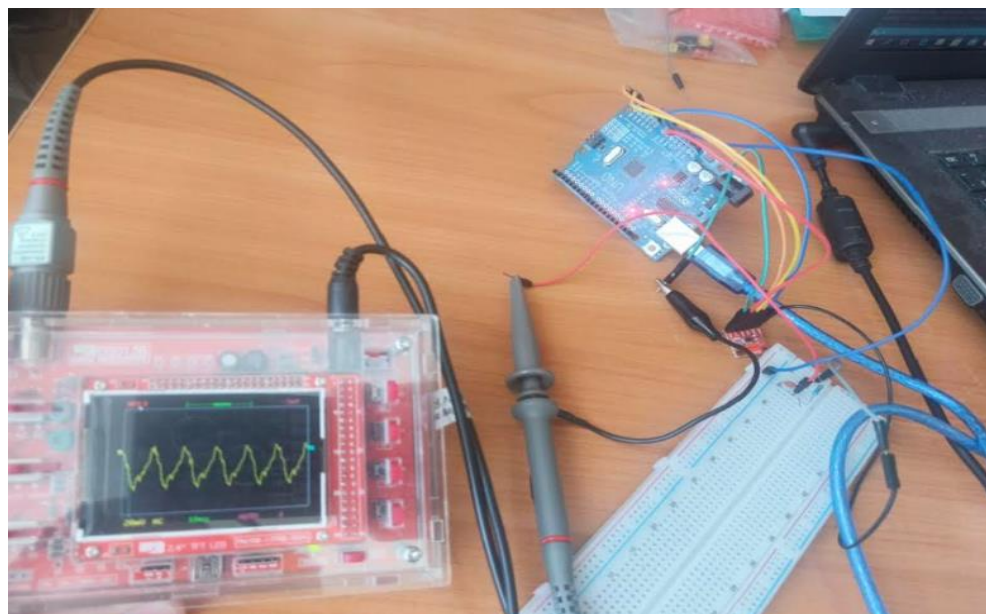


Рисунок 9 – Фотография макета генератора сигнала.

2.16 Согласование компонентов по напряжению и частоте

Микроконтроллерная плата Arduino работает от напряжения 5 В через USB. Согласно технической документации ЦАП MCP4725 работает на напряжении в диапазоне от 2,7 В до 5,5 В [16] ПРИЛОЖЕНИЕ Б Рисунок Б1. Данные компоненты совместимы с микроконтроллерной платой Arduino. Осциллограф имеет питание 9 В, что попадает в его диапазон питания согласно техническим характеристикам ПРИЛОЖЕНИЕ Б Таблица Б.1

Частота шины I2C программно установлена в 400 кГц. Согласно технической документации для ЦАП ПРИЛОЖЕНИЕ Б Рисунок Б1 и по расчётам передача одного отсчёта занимает 95 мкс. Частота дискретизации определена программно заданной задержкой в 1500 мкс.

$$F_{\text{Дискр}} = \frac{1}{0,015} = 667 \text{ Гц} \quad (2.2)$$

Согласно технической документации время на один отчёт у ЦАП равно 6 мкс. Значит его максимальная частота равна:

$$F_{\text{Макс_MCP4725}} = \frac{1}{0,000006} = 167 \text{ кГц} \quad (2.3)$$

Частота дискретизации меньше, чем максимальная частота ЦАП $667 \text{ Гц} < 167 \text{ кГц}$, Количество отсчётов на период задаётся N от 8 до 100 программно. При N=8 получен сигнал на выходе с частотой:

$$F_{\text{Сиг_вых}} = \frac{F_{\text{дискр}}}{N} \quad (2.4)$$

$$F_{\text{Сиг_вых}} = \frac{667}{8} = 83 \text{ Гц} \quad (2.5)$$

При N=100 получен сигнал на выходе с частотой:

$$F_{\text{Сиг_вых}} = \frac{667}{100} = 6,7 \text{ Гц} \quad (2.6)$$

Как видно из расчётов максимальная частота выходного сигнала равна 83 Гц, что гораздо меньше, чем 200 кГц – максимальная частота, которую осциллограф может обработать без искажений.

Для сглаживания высокочастотных импульсных помех на выходе ЦАП МСР4725 установлена RC-цепочка, состоящая из резистора номиналом 220 Ом и керамического конденсатора 683 (0,068 мкФ).

$$F_{\text{ср}} = \frac{1}{2\pi RC} \quad (2.9)$$

$$F_{\text{ср}} = \frac{1}{2 \cdot 3,14 \cdot 220 \cdot 0,0068} = 10,6 \text{ кГц}$$

Частота 10,6 кГц превышает значение максимальной частоты выходного сигнала (83 Гц). Фильтр подавляет высокочастотные шумы выше 10,6 кГц.

.

3 РЕАЛИЗАЦИЯ ПРОГРАММЫ, ВЫПОЛНЯЮЩЕЙ АЛГОРИТМ АДАПТИВНОГО КВАНТОВАНИЯ НА ЯЗЫКЕ ПРОГРАММИРОВАНИЯ C

3.1 Выбор языка программирования и среды разработки

В данной работе программа написана на языке программирования C. Этот язык программирования выбран из-за его высокой распространённости при работе с микроконтроллерами и быстродействия при работе с встраиваемыми системами. Программа написана на ПК в среде разработки Arduino IDE, загружается на микроконтроллерную плату и выполняется микроконтроллером. Среда разработки Arduino IDE выбрана, потому что в ней возможно использование готовых библиотек для интерфейса передачи данных I2C и ЦАП MCP4725, что позволило сократить время разработки, при этом сохраняя надёжность при передаче информации.

Готовые библиотеки контролируют временные интервалы интерфейса I2C и предотвращают программные сбои при обмене данными. Также выбор этой среды разработки обусловлен тем, что данная среда широко распространена при программировании микроконтроллерной платы Arduino.

Часть программы, выполняющая математические вычисления, более требовательные к памяти и мощности процессора устройства, реализована на ПК в среде разработки Visual Studio. Данная среда разработки является распространённой, доступной и бесплатной, поэтому выбор был остановлен на ней.

3.2 Структура программы

3.2.1 Подключение библиотек

Написание программы началось с подключения библиотек:

`#include <Wire.h>` – стандартная библиотека среды Arduino для обмена данными по интерфейсу I2C.

`#include <Adafruit_MCP4725.h>` – библиотека для работы с ЦАП MCP4725.

3.2.2 Объявление объектов констант, переменных

`Adafruit_MCP4725 dac` – создание объекта для управления ЦАП MCP4725.

`N_MAX=101` – ограничение в программе максимального размера массива для количества отсчётов на период.

Ограничиваясь числом 101, была получена экономия места в памяти микроконтроллера, не выделяя лишних ячеек памяти при задействовании ровно такого количества памяти, чтобы обработать весь массив начиная от значения 0 до значения 100 включительно, и соответствуя ограничению $val \leq 100$.

OFFSET=2048 – смещение в программе выбрано равным 2048, так как – это середина диапазона 12 битного ЦАП, который способен выдавать 4096 только положительный уровней. Если же часть сигнала будет отрицательной, то ЦАП сможет вывести только положительную часть сигнала. Для этого в программе используется смещение.

LED_PIN=13 – использование встроенного светодиода в плату для включения по окончании процесса вычисления алгоритма.

В программе задаются переменные:

X[N_MAX] – для хранения точек идеальной аналоговой синусоиды. XD2[N_MAX] – для хранения дискретных точек адаптированной синусоиды. Current_N – переменная задаёт текущее значение количества отсчётов на период.

Current_A – переменная задаёт текущее значение амплитуды сигнала.

need_recalc – переменная булевого типа, предназначенная для определения программой: нужно ли проводить вычисления по алгоритму. Несёт в себе значение true в начале выполнения программы и при изменении пользователем значений числа отсчётов на период и амплитуды.

current_point – переменная, несущая в себе порядковый номер элемента из массива XD2. А

active_cmd – переменная, принимающая символьное значение амплитуды и количества отсчётов на период из строки, введённые пользователем, для определения какой именно параметр нужно изменить в программе.

incoming_value – переменная, принимающая числовое значение амплитуды и количества отсчётов на период из строки, введённые пользователем.

3.2.3 Вспомогательные функции для работы с числами, функции process_signal и calculate_and_print_error()

После подключения библиотек в программе реализована функция Okrugli. Функция принимает дробное число (double). Проверяет полученное число является положительным или отрицательным.

В случае если, число положительное функция прибавляет к нему 0.5 и округляет путём преобразования полученного числа из дробного (double) в целочисленное (int). Например, получено число 3.8. Функция прибавит к этому числу 0.5. $3.8+0.5=4.3$ и округлит, отбросив дробную часть. Останется число 4.

В случае если, число отрицательное функция отнимает от него 0.5 и округляет путём преобразования полученного числа из дробного (double) в целочисленное (int).

Например, получено число -6.3. Функция прибавит к этому числу -6.3.-0.5=-6.8 и округлит, отбросив дробную часть. Останется число -6.

Эта функция предназначена для округления до ближайшего целого числа для чисел как положительных, так и отрицательных.

Далее в программе реализована функция Otbros. Эта функция получает число отбрасывает дробную часть преобразуя дробное число (double) в целочисленное (int). Полученное число как положительное, так и отрицательное всегда округляется в сторону 0.

Например, из числа 3.8 будет получено число 3. Из числа -6.3 после использования функции будет получено число -6. Эта функция нужна для получения нижнего значения для алгоритма (Nzn).

Функция Znak служит для проверки полученного дробного числа является положительным, отрицательным или равно нулю. В случае если число положительное, программа возвращает единицу. Если же число отрицательное программа вернёт минус один. Если число равно нулю, то программа возвращает ноль.

Функция calculate_and_print_error() вычисляет первую гармонику массива отсчётов по формулам дискретного преобразования Фурье. После чего данная функция производит расчёт относительной погрешности.

Эта функция вызывается два раза в process_signal() до адаптации сигнала и после для наглядного сравнения снижения относительной погрешности.

3.2.4 Блок формирования идеальной синусоиды

Далее идёт блок формирования сигнала, используемого как идеальный. Для формирования исходного сигнала используется расчёт массива идеальных значений сигнала с помощью цикла от 1 до N (количество точек дискретизации).

В цикле происходит вычисление переменной ugo1 (текущий фазовый угол) по формуле:

$$ugol = \frac{2 * \pi * i}{\frac{1.001}{N}} - \frac{\pi}{N} \quad (3.1)$$

1.001 – коэффициент, введённый для имитации погрешности частоты сигнала, нужный для того, чтобы синусоида имела период, не попадающий идеально в сетку оцифровки. Это приближает сигнал к реальным неидеальным условиям

Амплитуда каждой точки сигнала считается на основе переменной $ugol$ и записывается в массив $X[i]$.

$$X[i]=A*\sin(ugol) \quad (3.2)$$

где A - амплитуда в конкретной точке.

3.2.5 Блок квантования

Квантование тремя способами

Берутся дробные значения сигнала в определённых точках. Внутри цикла эти точки округляются разными способами, имитируя работу ЦАП.

Значения, округленные в меньшую сторону, записываются в массив XDN , в большую – в массив XDN . Значения, округленные до ближайшего целого числа, записываются в массив XDO . Массив $XD2$ является копией массива с округлением до ближайшего целого.

Блок для получения результатов сигнала с округлением до ближайшего значения в каждой точке и исходного.

В этом блоке происходит расчёт синусоидальной и косинусоидальной составляющих сигнала. $SumSinId$ и $SumCosId$ – это переменные, накапливающие данные для идеального сигнала. $SumSinOk$ и $SumCosOk$ – это переменные, накапливающие данные для округленного сигнала.

Поправочный коэффициент K нужен для компенсации погрешностей дискретизации. K рассчитывается по формуле:

$$K = \frac{2}{N} * \frac{\sin * \frac{\pi}{N}}{\frac{\pi}{N}} \quad (3.3)$$

Значение амплитуд основной гармоники для идеального и квантованного сигнала представляют из себя геометрическую сумму косинусной и синусной составляющей, умноженной на поправочный коэффициент K .

3.2.6 Блок, отвечающий за адаптивный цикл в программе.

1. Объявляется переменная `old_val`, обозначающая значение определённой точки сигнала, переменная `u` для расчёта угла внутри цикла `for`. С учётом угла рассчитывается синусная и косинусная составляющая точки.
2. Из суммы косинусной и суммы синусной составляющей вычитается значение текущей точки, чтобы понять, какие именно изменения даёт текущая точка на сигнал при разном способе округления.
3. Внутри цикла `for`, где происходит прохождение по каждой точке сигнала `i`. Внутри цикла текущее значение сначала округлится до ближайшего значения, затем округляется в верхнюю сторону, после чего – в нижнюю сторону.
4. Берётся сумма синусной и сумма косинусной составляющей без учёта текущей точки, к ним прибавляется соответствующая составляющая этой точки. Вычисляется амплитуда первой гармоники, с учётом одной точки округленной вверх через сумму квадратов синусной и косинусной составляющей, умноженной на поправочный коэффициент `K`. Сравнивается разница между амплитудой первой гармоники идеального сигнала и адаптированного сигнала. Аналогично для точки, округленной до нижнего и ближайшего значения.
5. Объявляется переменная `best_val`, в которую записывается значение округленной точки таким способом, что разница между амплитудой первой гармоники идеального и округленного сигнала будет минимальной.
6. В консоль Arduino IDE отправляется информация об окончании расчёта для данного количества отсчётов, первая гармоника идеальной синусоиды имеет такую-то амплитуду. Светодиод загорается, для индикации о завершении процесса вычислений. [9]

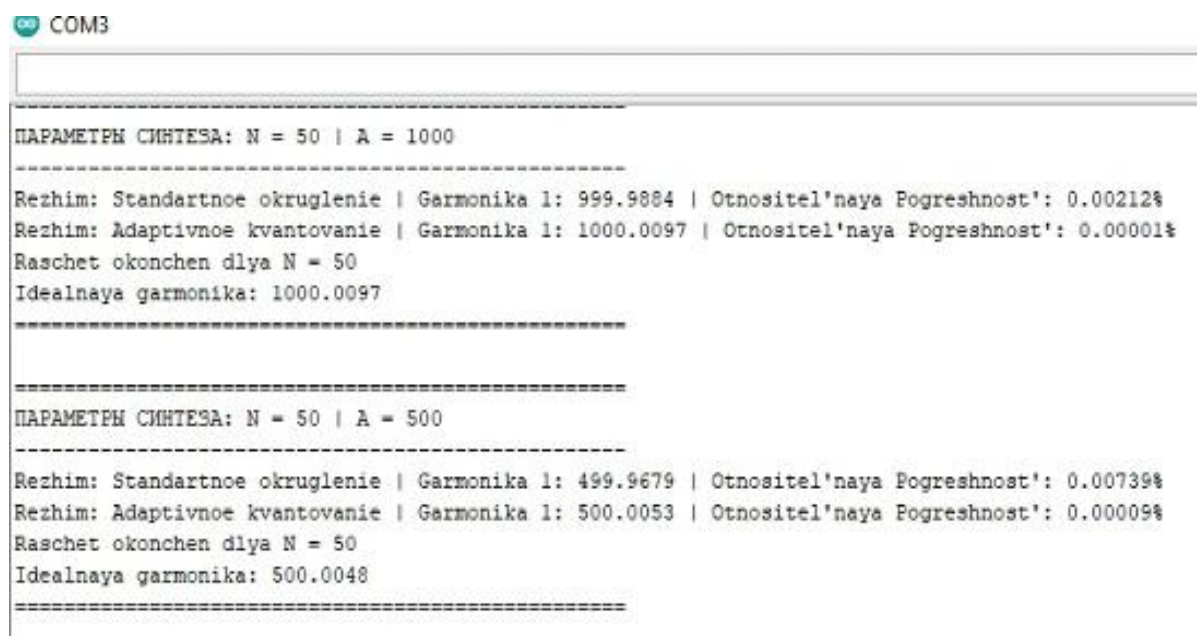
3.2.8 Функция `Setup` и `Loop`

В данной функции вывод светодиода переходит в режим `OUTPUT` и устанавливается в состояние `LOW`.

Устанавливается стандартный режим передачи данных между ПК и платой Arduino 9600 бод в программе для соответствия скорости в Arduino IDE. За ЦАП закрепляется стандартный адрес `0x62`, соответствующий адресу в библиотеке. Устанавливается частота передачи данных по шине I2C 400 кГц (режим `FAST MODE` для максимально возможной скорости передачи данных). В консоли Arduino IDE появляется надпись о начале расчёта.

Далее идёт бесконечно выполняющаяся функция void loop. Сначала в функции идёт блок, где проверяется: введена ли пользователем информация в консоль. Если введена, то идёт проверка, что именно введено. Количество отсчётов или амплитуда, какое число введено. Соответствует ли число требуемому диапазону количества отсчётов на период и диапазону возможной амплитуды. Затем идёт блок проверки: нужен ли перерасчёт, выполняемый один раз при изменении параметров. После чего идёт блок с выводом значения точки из массива с точками адаптированного сигнала на ЦАП.

В данной бесконечной функции выведутся все точки одного периода на ЦАП и период будет бесконечно повторяться пока пользователь не изменит параметры. На Рисунок 10 представлен снимок экрана, демонстрирующий данные о количестве отсчётов на период, амплитуде и относительной погрешности, выведенные на COM порт. Блок-схема данной программы представлена на Рисунок 11.



```
COM3

ПАРАМЕТРЫ СИНТЕЗА: N = 50 | A = 1000
-----
Rezhim: Standartnoe okruglenie | Garmonika 1: 999.9884 | Otnositel'naya Pogreshnost': 0.00212%
Rezhim: Adaptivnoe kvantovanie | Garmonika 1: 1000.0097 | Otnositel'naya Pogreshnost': 0.00001%
Raschet okonchen dlya N = 50
Idealnaya garmonika: 1000.0097
-----

ПАРАМЕТРЫ СИНТЕЗА: N = 50 | A = 500
-----
Rezhim: Standartnoe okruglenie | Garmonika 1: 499.9679 | Otnositel'naya Pogreshnost': 0.00739%
Rezhim: Adaptivnoe kvantovanie | Garmonika 1: 500.0053 | Otnositel'naya Pogreshnost': 0.00009%
Raschet okonchen dlya N = 50
Idealnaya garmonika: 500.0048
-----
```

Рисунок 10 – Снимок экрана, демонстрирующий данные о количестве отсчётов на период, амплитуде и относительной погрешности, выведенные на COM порт.

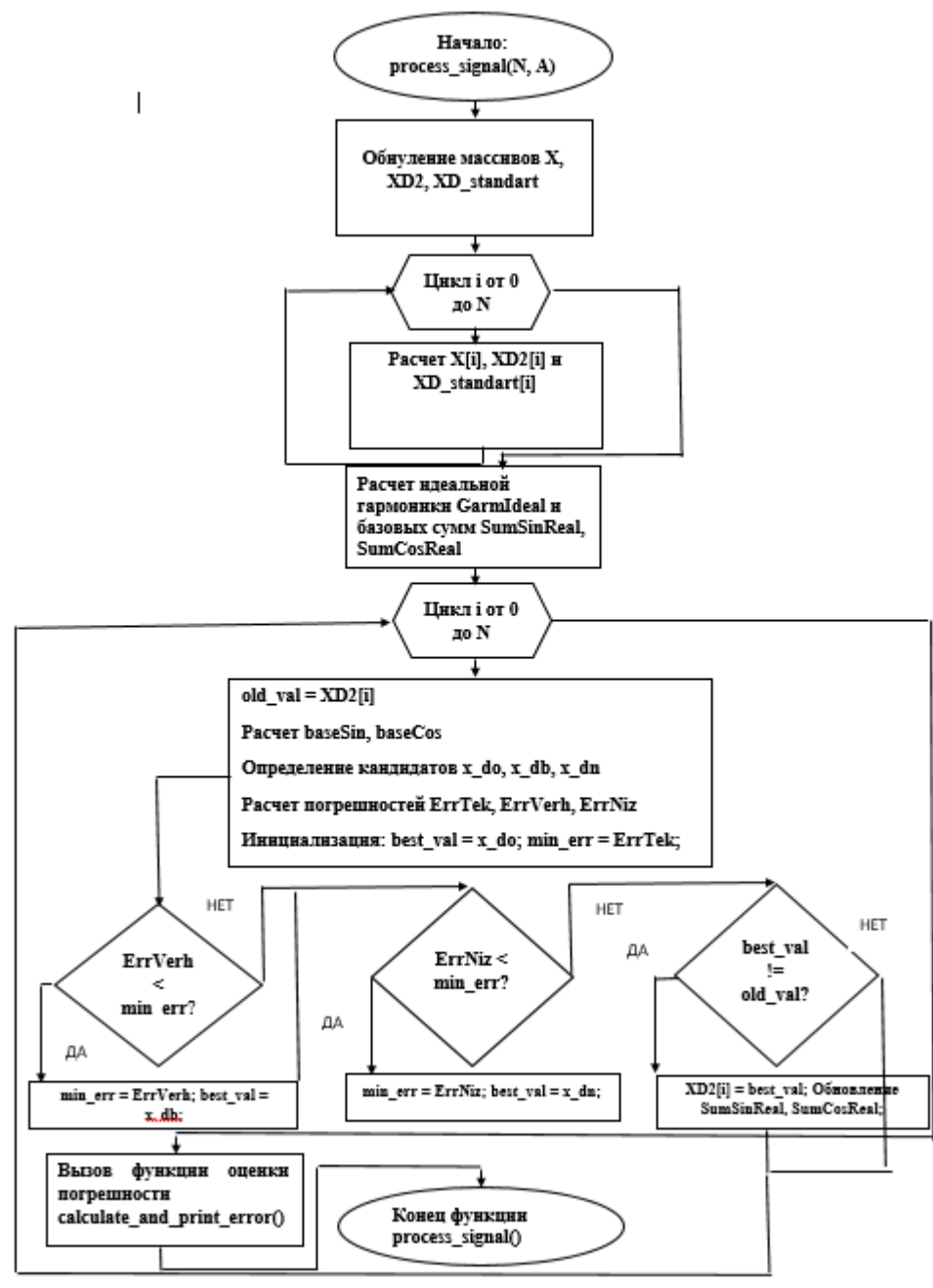


Рисунок 11 – Блок-схема программы, выполненной в среде разработки Arduino IDE

3.2.9 Программа для расчёта на персональном компьютере

Для детального анализа эффективности алгоритма адаптивного квантования написана отдельная программа на языке C, выполняемая на ПК в Visual Studio. Так как ПК, в отличие от микроконтроллерной платы не ограничен по памяти настолько сильно, это позволяет программе провести расчёты для количества отсчётов с шагом 2, потом 10, 20 и так до 100 и при значении амплитуды 200, 500, 1000, 1500, 2000.

Для каждого сочетания количества отсчётов и амплитуды программа рассчитывает четыре метода округления: только вверх, только вниз, до ближайшего значения и адаптивное квантование.

По каждому методу рассчитывается относительная погрешность первой гармоники и относительная погрешность СКЗ. Все результаты программы сохраняются в текстовый файл. Первые строчки файла приведены в Приложении Г. Блок-схема программы, реализованной в Visual Studio представлена на Рисунок 12-14. На основе данных из файла нарисованы графики зависимости погрешности первой гармоники от числа отсчётов при разной амплитуде сигнала, приведённые на изображениях Рисунок 15 – Рисунок 17, и графики зависимости погрешности СКЗ от числа отсчётов при разной амплитуде сигнала на изображениях Рисунок 18 – Рисунок 20. Первые страницы файла приведены в ПРИЛОЖЕНИЕ Г РИСУНОК Г.1

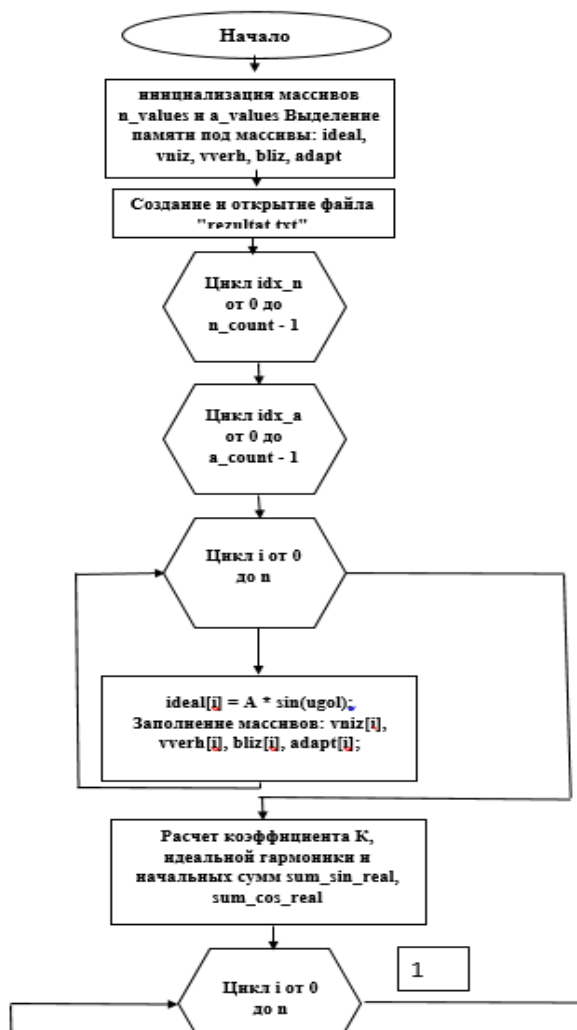


Рисунок 12 – Блок-схема программы, реализованной в Visual Studio

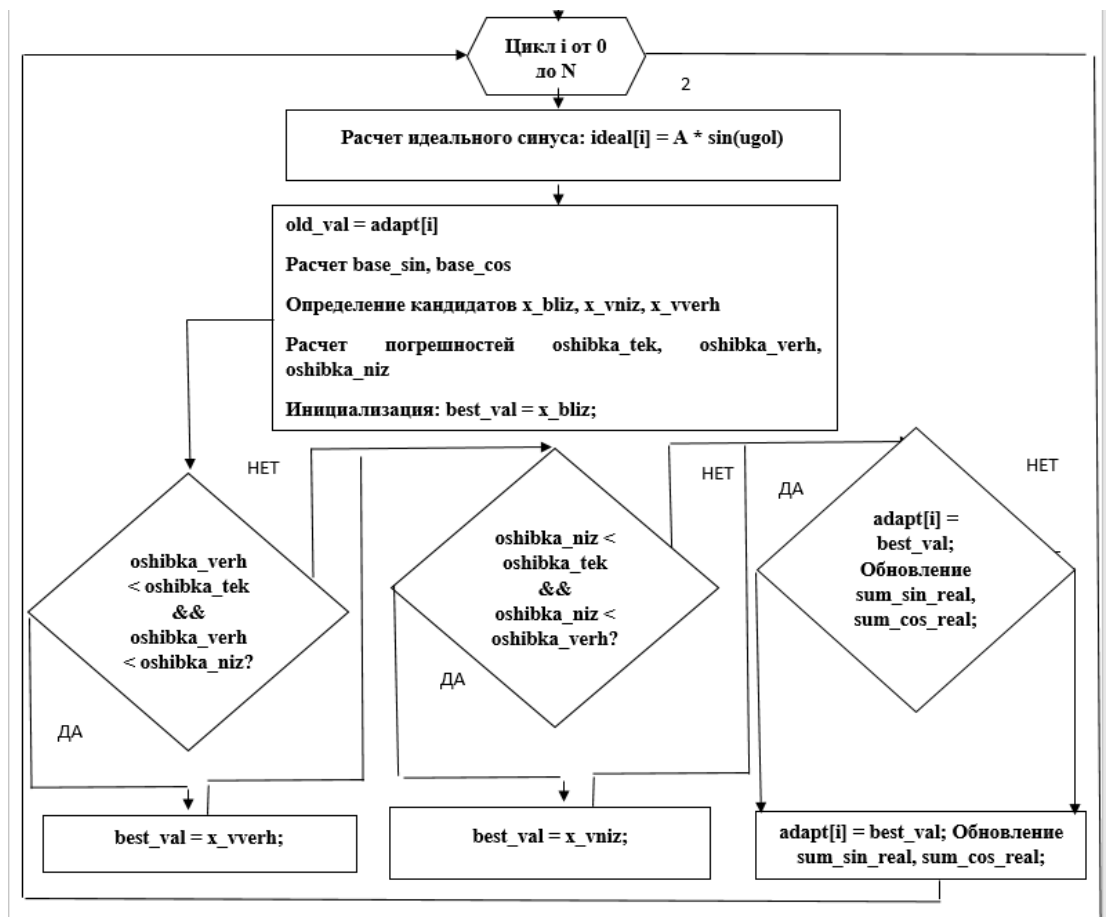


Рисунок 13 – Блок-схема программы, реализованной в Visual Studio

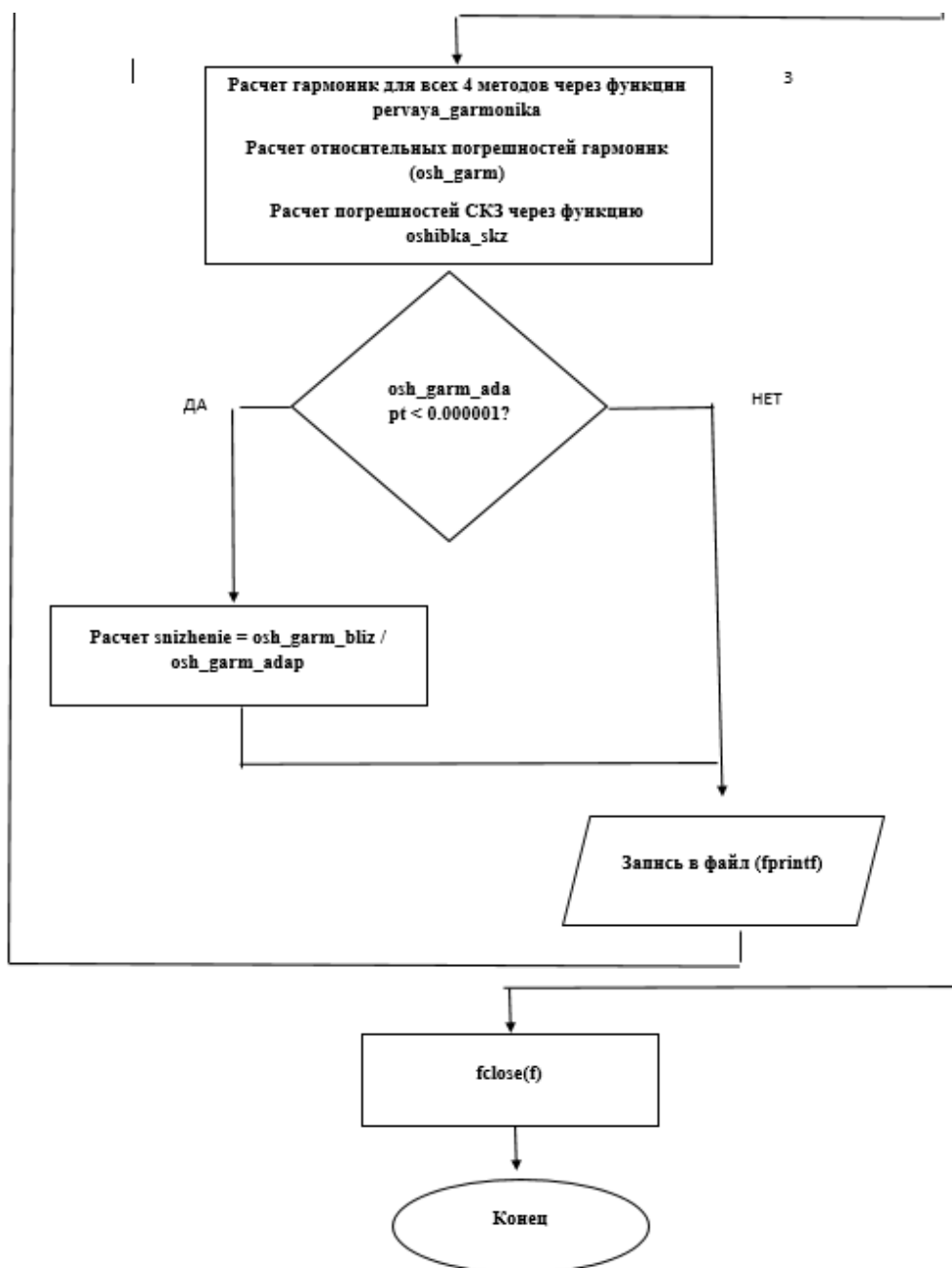


Рисунок 14 - Блок схема программы, реализованной в Visual Studio

Так как блок-схема имеет большой объём и разделена на несколько рисунков линии возврата вложенных циклов указаны текстом:

1. После записи данных (блок fprintf) управление возвращается к выбору следующей амплитуды (Цикл idx_a).
2. После перебора всех амплитуд алгоритм переходит к следующему количеству точек (Цикл idx_n).
3. При завершении всех итераций (idx_n) выполняется закрытие файла (fclose) и выход из программы.

4. После проверки условия ($best_val \neq old_val?$) поток данных возвращается на начало адаптивного Цикла i .

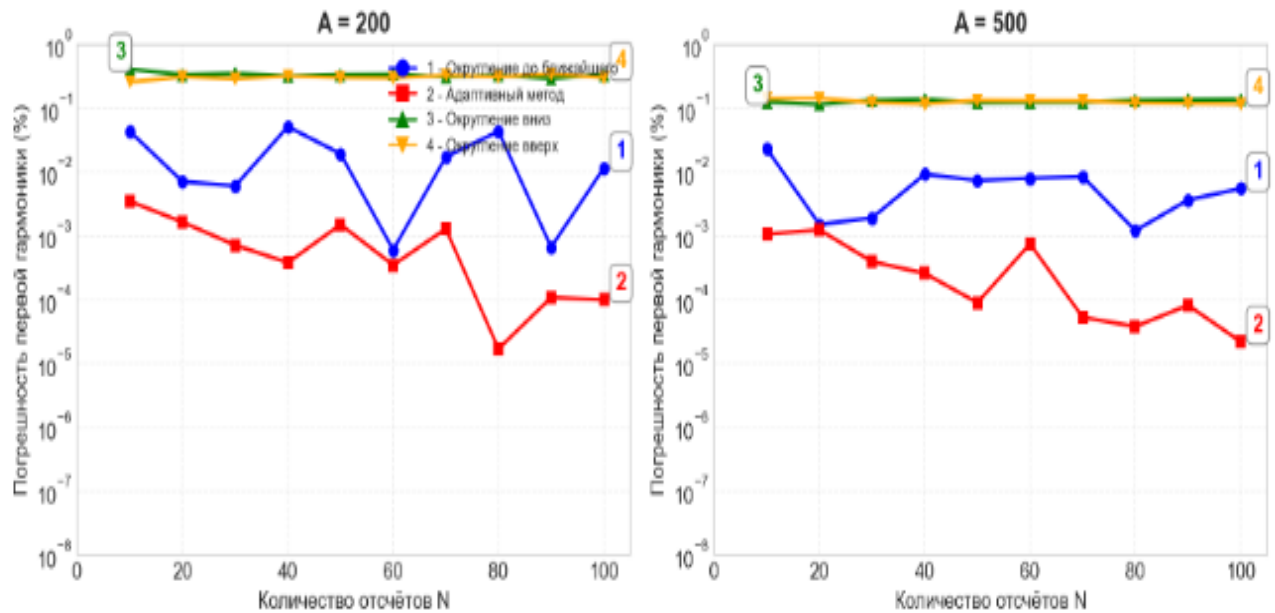


Рисунок 15 - график зависимости погрешности первой гармоники от числа отсчётов при амплитуде $A=200$, $A=500$

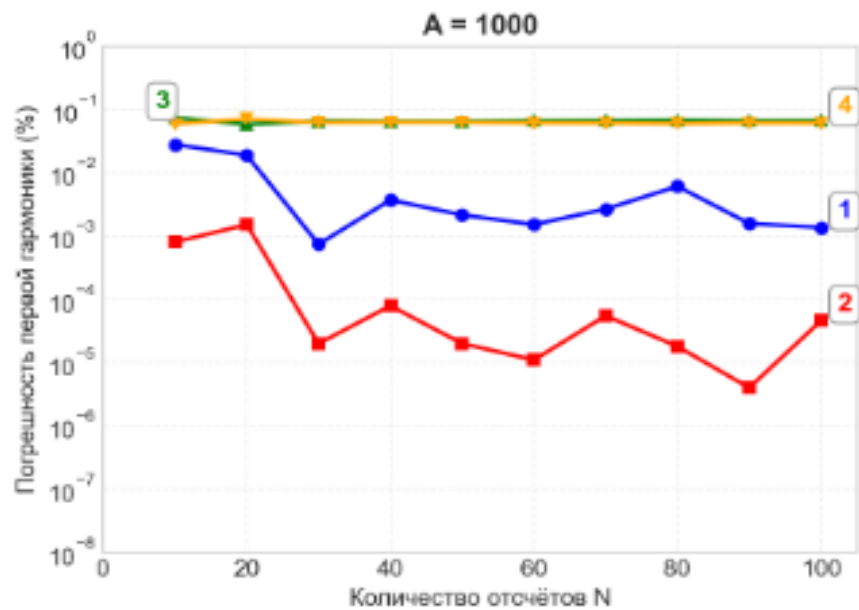


Рисунок 16 - график зависимости погрешности первой гармоники от числа отсчётов при амплитуде $A=1000$

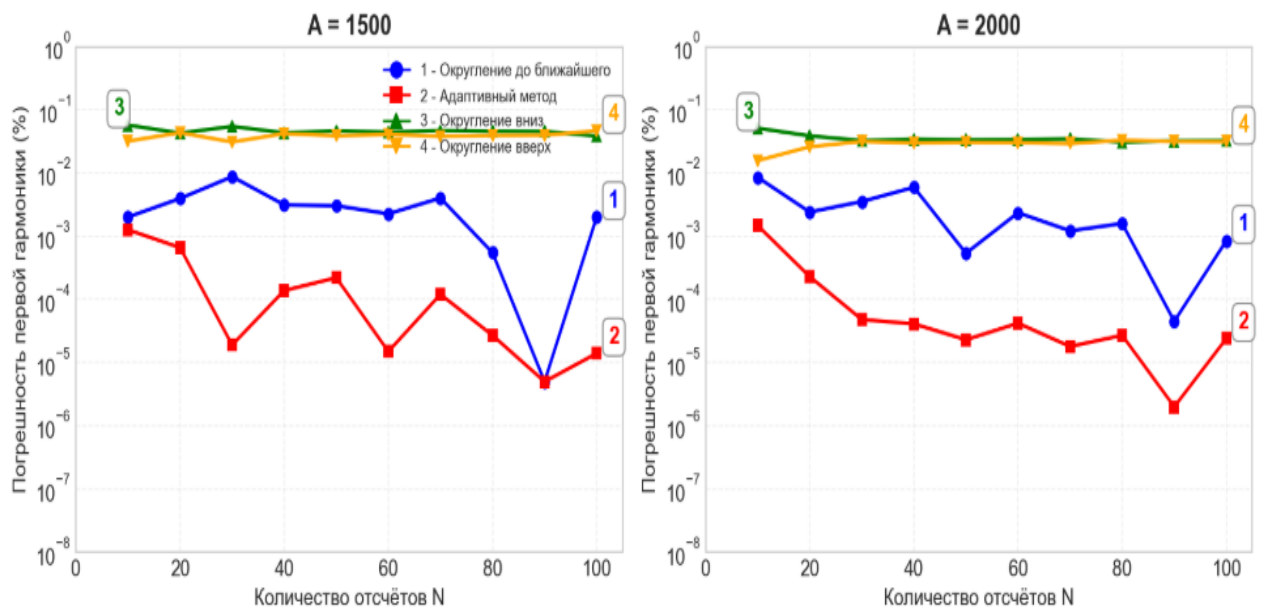


Рисунок 17 - график зависимости погрешности первой гармоники от числа отсчётов при амплитуде $A=1500, 2000$

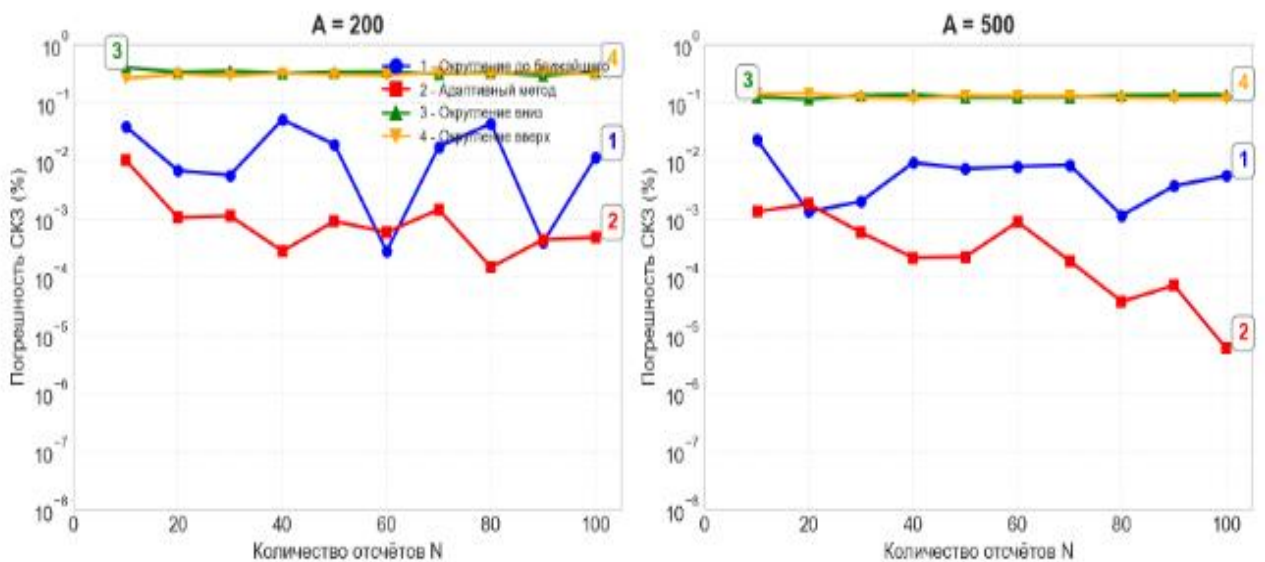


Рисунок 18 - график зависимости погрешности СКЗ от числа отсчётов при амплитуде $A=200, A=500$

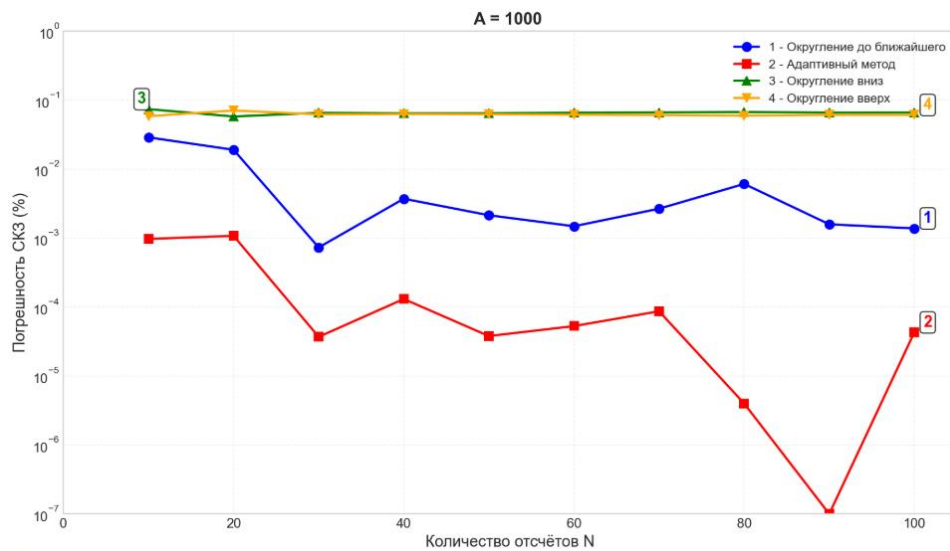


Рисунок 19 - график зависимости погрешности СКЗ от числа отсчётов при амплитуде $A=1000$

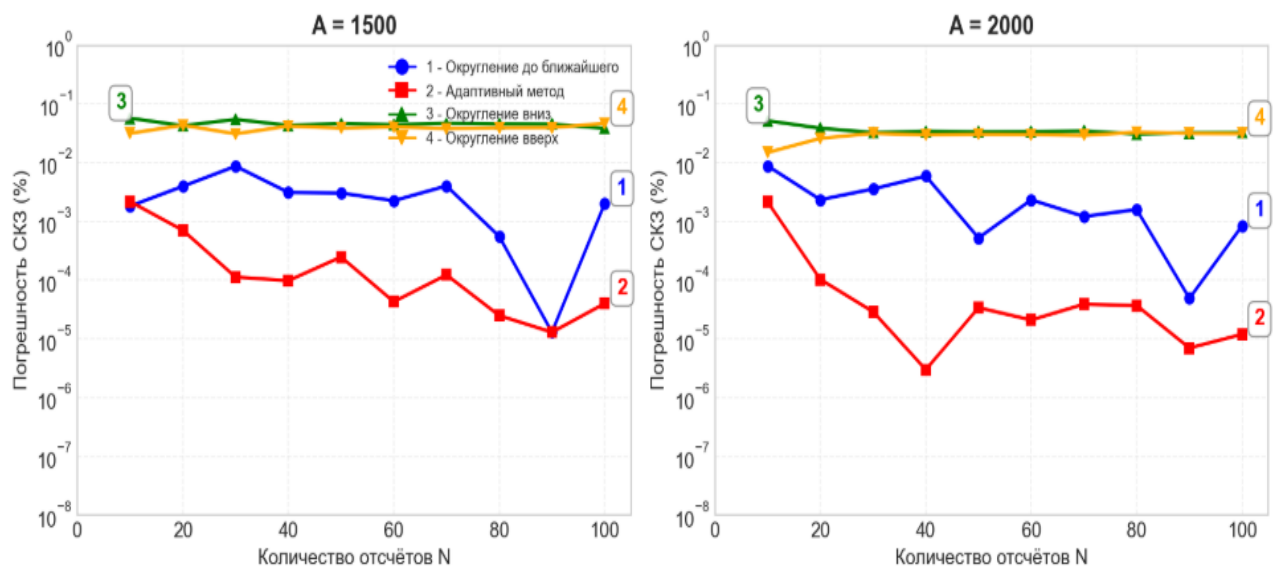


Рисунок 20 - график зависимости погрешности СКЗ от числа отсчётов при амплитуде $A=1500, 2000$

Анализ графиков, построенных по результатам расчётов, показал следующее. Округление только вверх или только вниз даёт погрешность намного выше, чем округление до ближайшего уровня. Адаптивное квантование на всех графиках либо снижает относительную погрешность первой гармоники по сравнению с другими способами квантования, либо работает не хуже, чем округление до ближайшего значения. По критерию СКЗ адаптивное квантование в большинстве случаев даёт результат не хуже других методов округления, а в редких случаях незначительно уступает (в пределах сотых долей процента). Незначительное ухудшение СКЗ объясняется перераспределением энергии сигнала в область высших гармоник — это плата за улучшение первой гармоники, что соответствует теоретическим данным из статьи [4]. Высшие гармоники могут быть отфильтрованы.

4 ЭКОНОМИЧЕСКАЯ ЧАСТЬ

Использование интерфейса передачи данных I2C позволяет использовать относительно других возможных интерфейсов малое количество проводов, что позволяет сэкономить на проводах и оставить свободные выводы у микроконтроллерной платы, что уже позволит выбрать более дешевую плату с меньшим количеством выводов.

Существование готовых библиотек для использования ЦАП MCP4725 с Arduino позволяет сэкономить на количестве рабочих часов программиста при написании логики управления шиной на низком уровне.

Использование данного модуля ЦАП обусловлено его низкой ценой, распространённостью и доступностью.

В работе использовался портативный осциллограф DSO138 так как его стоимость намного ниже, чем стационарной модели.

Микроконтроллерная плата Arduino Uno обладает невысокой стоимостью при достаточных способностях микроконтроллера для осуществления выполнения работы по реализации алгоритма адаптивного квантования.

Стоимость каждого элемента схемы по отдельности и в сумме представлена в Таблице 4.

Таблица 4 – Стоимость элементов, используемых в схеме устройства для реализации алгоритма адаптивного квантования

Название компонента	Цена (руб.)
Arduino Uno (копия китайского производства с характеристиками, как у оригинала, в комплекте: USB-кабель, резисторы, конденсаторы)	573
ЦАП MCP4725	230
Макетная плата	243
Набор проводов	241
Осциллограф DSO138	1793
Полная стоимость	3080

5 ОХРАНА ТРУДА

Создание программы, выполняющей алгоритм адаптивного квантования происходило за работой с персональным компьютером. Реализация практической составляющей выпускной квалификационной работы было связано с подключением модуля ЦАП к микроконтроллерной плате.

5.1 Режим труда и отдыха

Для отдыха и разминки глаз и опорно-двигательного аппарата каждые 20 минут работа прерывалась на короткую разминку.

Длительность непрерывного нахождения у монитора компьютера была ограничена двумя часами.

Общая длительность нахождения перед монитором компьютера в сутки была ограничена 8 часами.

5.2 Общие требования к организации рабочего пространства

1. Экран монитора компьютера должен быть расположен на расстоянии 60-70 см от глаз пользователя.
2. Площадь одного рабочего места с компьютером должно быть не менее 4,5 м².
3. Уровень освещённости рабочего места должен быть не менее 300 люкс.
4. Стул должен позволять сохранять физиологически правильную позу.

6 ИМПОРТОЗАМЕЩЕНИЕ

В данной работе в качестве микроконтроллерной платы была выбрана из-за доступности и низкой стоимости плата китайского производства, с такими же характеристиками, как и у оригинала. Стоит отметить, что существуют микроконтроллерные платы российского производства:

1. Плата ACE-UNO от компании «Микрон», построенная на базе ядра «Амур» (МИК32 АМУР K1933BK018). Цена 7000-8000 рублей.
2. Плата «ИвМикро» с микроконтроллером серии K1921BK (производитель АО «НИИЭТ»). Цена 3000-4000 рублей.
3. Возможно использование одноплатного компьютера Рерка Pi 3 (1 Гб ОЗУ). Его производитель – это компания ООО «НПО СВТ», сам компьютер собирается в РФ, чип производится в КНР. Цена 7000-11 000 рублей.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы достигнута поставленная цель: написан и экспериментально подтверждён алгоритм адаптивного квантования на языке программирования C. Эксперимент выполнен на микроконтроллерной плате Arduino Uno с использованием ЦАП MCP4725 интерфейса I2C, осциллографа DSO138.

Для исследования был изучен алгоритм адаптивного квантования по методу из патента №2420799. Выполняемая программа создана с опорой на уже существующую программу, написанную на языке Basic М. М. Бабичевым. Программа на C является усовершенствованной и расширенной, так как быстрее в компиляции за счёт языка программирования, выполняет расчёт для двух критериев оптимальности, а не одного при разном количестве отсчётов на период и разных амплитудах, проверяет 4 варианта округления, а не два. Алгоритм основывался на последовательном переборе вариантов округления каждого отсчёта. Разработанная программа на языке C выполняла последовательную проверку округления до ближайшего целого, округления верхнего значения, округления до нижнего. Каждое решение выбиралось в соответствии с подбором лучшего значения по критерию оптимальности.

Форма сигнала на экране осциллографа отличается от идеальной синусоиды: наблюдаются ступеньки и незначительные шумы. Это связано с наводками от схемы и ступенчатостью на выходе любого ЦАП. Адаптивное квантование не устраняет ступеньки, а оптимизирует их высоту для снижения погрешности первой гармоники и СКЗ, минимизация которых выбрана в качестве критериев оптимальности. Данные искажения не влияют на доказательство эффективности адаптивного квантования, поскольку расчёт погрешностей первой гармоники и СКЗ выполняется математически на основе цифровых отсчётов, а не по форме сигнала на экране. На основе данных, полученных программой, построены графики. Анализ показал: при округлении только вверх или только вниз погрешность выше, чем округление до ближайшего уровня. Адаптивное квантование в большинстве случаев снижает относительную погрешность первой гармоники и СКЗ по сравнению с округлением до ближайшего. Диапазон снижения составляет от 1 раза (при очень малом количестве отсчётов, например $N = 2$) до нескольких сотен раз (при $N > 20$ чаще всего снижение составляет от 10 до 150). Реализованный алгоритм адаптивного квантования доказал свою эффективность.

Практическая ценность работы заключается в возможности улучшения качества сигнала без замены ЦАП на более высокоразрядный и без добавления в схему программируемых усилителей, что позволяет снизить стоимость устройства.

СПИСОК ЛИТЕРАТУРЫ

1. ГОСТ 8.584-2004. Государственная система обеспечения единства измерений. Счётчики статические активной электрической энергии переменного тока. Методика поверки. – М. : ИПК Издательство стандартов, 2004. – 20 с.
2. Аналогово-цифровое преобразование / У. Кестер [и др.] ; под ред. У. Кестера ; пер. с англ. – М. : Техносфера, 2007. – 1016 с.
3. Баби́чев, М. М. Оценка погрешности дискретности при перемножении двух дискретных сигналов, заданных в виде кодов / М. С. Баби́чев ; науч. рук. Ю. А. Пасынков // Наука. Технологии. Инновации : сборник научных трудов. – Новосибирск : Изд-во НГТУ, 2015. – Ч. 1. – С. 22–24.
4. Баби́чев, М. М. Применение адаптивного квантования в измерительных генераторах с цифровым управлением / М. М. Баби́чев, В. А. Трушин // Научный вестник НГТУ. – 2018. – Т. 70, № 1. – С. 51–66.
5. Баби́чев, М. М. Реализация метода адаптивного квантования в измерительных генераторах с цифровым управлением / М. М. Баби́чев // Системы анализа и обработки данных. – 2021. – Т. 82, № 2. – С. 121–134.
6. Баскаков, С. И. Радиотехнические цепи и сигналы : учебник для вузов / С. И. Баскаков. – 5-е изд., стер. – М. : Высшая школа, 2005. – 462 с.
7. Быков, С. В. Цифровые интерфейсы измерительных устройств. Интерфейс I2C : учебное пособие к лабораторному практикуму / С. В. Быков, М. А. Савиных. – Новосибирск : Изд-во НГТУ, 2025. – 13 с.
8. Кудрявцев, К. Я. Методы оптимизации : учебное пособие для вузов / К. Я. Кудрявцев. – 2-е изд., перераб. и доп. – М. : Издательство Юрайт, 2025. – 114 с. – (Высшее образование).
9. Микроконтроллерная плата Arduino Uno R3. Руководство пользователя [Электронный ресурс] : техническая документация. – URL: <https://hcomp.ru> (дата обращения: 29.04.2026).
10. Монк, С. Програмируем Arduino: основы работы со скетчами / Саймон Монк ; пер. с англ. под ред. И. В. Русяевой. – 2-е изд. – М. : Лаборатория знаний, 2017. – 176 с.
11. Набор для сборки осциллографа DSO 138. Руководство пользователя. Версия 01 [Электронный ресурс] / JYE Tech Ltd ; пер. на рус. яз. Supereyes. – URL: <https://supereyes.ru> (дата обращения: 08.05.2026).
12. Оценки погрешности при измерении среднеквадратических значений периодических сигналов / М. М. Баби́чев, Ю. А. Пасынков // Актуальные проблемы электронного приборостроения (АПЭП–2014) = Actual problems of electronic instrument engineering (APEIE–2014) : труды XII международной конференции, Новосибирск, 2–4 октября 2014 г. : в 7 т. – Новосибирск : Изд-во НГТУ, 2014. – Т. 3. – С. 191–194.
13. Пат. 2420799 Российская Федерация, МПК G06F 17/7. Способ округления кодов функции / М. М. Баби́чев, Ю. А. Пасынков ; заявитель и патентообладатель Новосибирский государственный технический университет. – № 2009120115/08 ; заявл. 27.05.2009 ; опубл. 10.06.2011, Бюл. № 16. – 12 с.
14. Проектирование систем цифровой и смешанной обработки сигналов / У. Кестер, Дж. Брайэнт, Д. Кинг [и др.] ; ред. ориг. изд. У. Кестер ; пер. с англ. под ред. А. А. Власенко. – М. : Техносфера, 2010. – 326 с.

15. Сергиенко, А. Б. Цифровая обработка сигналов : учебник для вузов / А. Б. Сергиенко. – СПб. : Питер, 2011. – 558 с.
16. MCP4725. 12-Bit Digital-to-Analog Converter with EEPROM Memory [Electronic resource] : datasheet / Microchip Technology Inc. – 2009. – URL: <https://chipdip.ru> (дата обращения: 23.04.2026).

ПРИЛОЖЕНИЕ А

```
#include <Wire.h>
#include <Adafruit_MCP4725.h>

Adafruit_MCP4725 dac;

const int N_MAX = 101;
const int OFFSET = 2048;
const int LED_PIN = 13;

double X[N_MAX];
int XD2[N_MAX];
int XD_standart[N_MAX];

int current_N = 50;
double current_A = 2000.0;
bool need_recalc = true;

int current_point = 0;

char active_cmd = '\0';
int incoming_value = 0;

int Okrugli(double x) {
    int res_okr = 0;
    if (x >= 0) {
        double temp1 = x + 0.5;
        res_okr = (int)temp1;
    } else {
        double temp2 = x - 0.5;
        res_okr = (int)temp2;
    }
    return res_okr;
}

int Otbros(double x) {
    int celoe = (int)x;
    return celoe;
}

int Znak(double x) {
    if (x > 0) return 1;
    if (x < 0) return -1;
    return 0;
}

void process_signal(int N, double A) {
    digitalWrite(LED_PIN, LOW);

    for (int i = 0; i < N_MAX; i++) {
        X[i] = 0.0;
        XD2[i] = 0;
```

```

XD_standart[i] = 0;
}

double h_step = 1.001 * N;
for (int i = 0; i <= N; i += 1) {
    double ugol = 2 * PI * i / h_step - PI / N;
    X[i] = A * sin(ugol);
    XD2[i] = Okrugli(X[i]);
    XD_standart[i] = XD2[i];
}

double K = (2.0 / N) * sin(PI / N) / (PI / N);

double SumSinId = 0.0;
double SumCosId = 0.0;
for (int j = 0; j <= N; j += 1) {
    double ugol = (2 * j + 1) * PI / N;
    SumSinId += X[j] * sin(ugol);
    SumCosId += X[j] * cos(ugol);
}
double GarmIdeal = sqrt(SumSinId * SumSinId + SumCosId * SumCosId) * K;

Serial.println("=====");
Serial.print("ПАРАМЕТРЫ СИМТЕЗА: N = "); Serial.print(N);
Serial.print(" | A = "); Serial.println((int)A);
Serial.println("-----");
calculate_and_print_error(N, GarmIdeal, K, XD_standart, "Standartnoe okruglenie");

double SumSinReal = 0.0;
double SumCosReal = 0.0;
for (int j = 0; j <= N; j += 1) {
    double ugol = (2 * j + 1) * PI / N;
    SumSinReal += XD_standart[j] * sin(ugol);
    SumCosReal += XD_standart[j] * cos(ugol);
}

for (int i = 0; i <= N; i += 1) {
    int old_val = XD2[i];
    double u = (2 * i + 1) * PI / N;
    double sin_u = sin(u);
    double cos_u = cos(u);

    double baseSin = SumSinReal - (old_val * sin_u);
    double baseCos = SumCosReal - (old_val * cos_u);

    int x_do = Okrugli(X[i]);
    int x_db = Otbros(X[i]) + Znak(X[i]);
    int x_dn = Otbros(X[i]);

    double termVerhSin = baseSin + x_db * sin_u;
    double termVerhCos = baseCos + x_db * cos_u;
    double GarmVerh = sqrt(termVerhSin * termVerhSin + termVerhCos * termVerhCos) * K;

```

```

double ErrVerh = fabs(GarmVerh - GarmIdeal);

double termNizSin = baseSin + x_dn * sin_u;
double termNizCos = baseCos + x_dn * cos_u;
double GarmNiz = sqrt(termNizSin * termNizSin + termNizCos * termNizCos) * K;
double ErrNiz = fabs(GarmNiz - GarmIdeal);

double termTekSin = baseSin + x_do * sin_u;
double termTekCos = baseCos + x_do * cos_u;
double GarmTek = sqrt(termTekSin * termTekSin + termTekCos * termTekCos) * K;
double ErrTek = fabs(GarmTek - GarmIdeal);

int best_val = x_do;
double min_err = ErrTek;

if (ErrVerh < min_err) {
    min_err = ErrVerh;
    best_val = x_db;
}
if (ErrNiz < min_err) {
    min_err = ErrNiz;
    best_val = x_dn;
}

if (best_val != old_val) {
    XD2[i] = best_val;
    SumSinReal = baseSin + best_val * sin_u;
    SumCosReal = baseCos + best_val * cos_u;
}
}

calculate_and_print_error(N, GarmIdeal, K, XD2, "Adaptivnoe kvantovanie");

Serial.print("Raschet okonchen dlya N = ");
Serial.println(N);
Serial.print("Idealnaya garmonika: ");
Serial.println(GarmIdeal, 4);
Serial.println("=====");
Serial.println();

digitalWrite(LED_PIN, HIGH);
}

void setup() {
    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    Serial.begin(9600);

    dac.begin(0x62);
    Wire.setClock(400000);

```

```

Serial.println("--- СИСТЕМА ГОТОВА К РУЧНОМУ ВВОДУ КОМАНД ---");
}

void loop() {
  if (Serial.available() > 0) {
    char ch = Serial.read();

    if (ch == 'n' || ch == 'N' || ch == 'a' || ch == 'A') {
      active_cmd = ch;
      incoming_value = 0;
    }
    else if (ch >= '0' && ch <= '9') {
      int digit = ch - '0';
      incoming_value = (incoming_value * 10) + digit;
    }
    else if (ch == '\n' || ch == '\r' || ch == ' ' || ch == ';') {
      if (active_cmd == 'n' || active_cmd == 'N') {
        if (incoming_value >= 8 && incoming_value <= 100) {
          current_N = incoming_value;
          need_recalc = true;
        }
      }
      else if (active_cmd == 'a' || active_cmd == 'A') {
        if (incoming_value >= 100 && incoming_value <= 2047) {
          current_A = (double)incoming_value;
          need_recalc = true;
        }
      }
      active_cmd = '\0';
    }
  }

  if (need_recalc) {
    process_signal(current_N, current_A);
    current_point = 0;
    need_recalc = false;
  }

  int out_val = XD2[current_point] + OFFSET;

  if (out_val > 4095) out_val = 4095;
  if (out_val < 0) out_val = 0;

  dac.setVoltage(out_val, false);

  current_point += 1;
  if (current_point > current_N) {
    current_point = 0;
  }

  delayMicroseconds(1500);
}

```

```

void calculate_and_print_error(int N, double GarmIdeal, double K, int* signal_array, const char*
mode_name) {
    double SumSinReal = 0.0;
    double SumCosReal = 0.0;

    for (int j = 0; j <= N; j += 1) {
        double ugol = (2 * j + 1) * M_PI / (double)N;
        SumSinReal += (double)signal_array[j] * sin(ugol);
        SumCosReal += (double)signal_array[j] * cos(ugol);
    }

    double GarmReal = sqrt(SumSinReal * SumSinReal + SumCosReal * SumCosReal) * K;
    double relative_error = (fabs(GarmIdeal - GarmReal) / GarmIdeal) * 100.0;

    Serial.print("Rezhim: ");
    Serial.print(mode_name);
    Serial.print(" | Garmonika 1: ");
    Serial.print(GarmReal, 4);
    Serial.print(" | Otnositel'naya Pogreshnost': ");
    Serial.print(relative_error, 5);
    Serial.println("%");
}

```

ПРИЛОЖЕНИЕ Б

Техническая документация элементов
Техническая документация ЦАП МСР 4725 указана в Рисунках 1-7.

MICROCHIP

MCP4725

12-Bit Digital-to-Analog Converter with EEPROM Memory in SOT-23-6

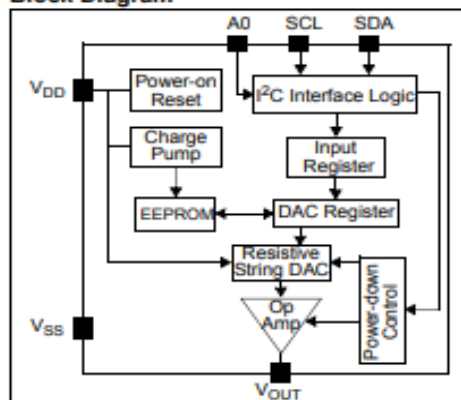
Features

- 12-Bit Resolution
- On-Board Non-Volatile Memory (EEPROM)
- ± 0.2 LSB DNL (typical)
- External A0 Address Pin
- Normal or Power-Down Mode
- Fast Settling Time: 6 μ s (typical)
- External Voltage Reference (V_{DD})
- Rail-to-Rail Output
- Low Power Consumption
- Single-Supply Operation: 2.7V to 5.5V
- I²C™ Interface:
 - Eight Available Addresses
 - Standard (100 kbps), Fast (400 kbps), and High-Speed (3.4 Mbps) Modes
- Small 6-lead SOT-23 Package
- Extended Temperature Range: -40°C to +125°C

Applications

- Set Point or Offset Trimming
- Sensor Calibration
- Closed-Loop Servo Control
- Low Power Portable Instrumentation
- PC Peripherals
- Data Acquisition Systems

Block Diagram



DESCRIPTION

The MCP4725 is a low-power, high accuracy, single channel, 12-bit buffered voltage output Digital-to-Analog Converter (DAC) with non-volatile memory (EEPROM). Its on-board precision output amplifier allows it to achieve rail-to-rail analog output swing.

The DAC input and configuration data can be programmed to the non-volatile memory (EEPROM) by the user using I²C interface command. The non-volatile memory feature enables the DAC device to hold the DAC input code during power-off time, and the DAC output is available immediately after power-up. This feature is very useful when the DAC device is used as a supporting device for other devices in the network.

The device includes a Power-On-Reset (POR) circuit to ensure reliable power-up and an on-board charge pump for the EEPROM programming voltage. The DAC reference is driven from V_{DD} directly. In power-down mode, the output amplifier can be configured to present a known low, medium, or high resistance output load.

The MCP4725 has an external A0 address bit selection pin. This A0 pin can be tied to V_{DD} or V_{SS} of the user's application board.

The MCP4725 has a two-wire I²C™ compatible serial interface for standard (100 kHz), fast (400 kHz), or high speed (3.4 MHz) mode.

The MCP4725 is an ideal DAC device where design simplicity and small footprint is desired, and for applications requiring the DAC device settings to be saved during power-off time.

The device is available in a small 6-pin SOT-23 package.

Package Type

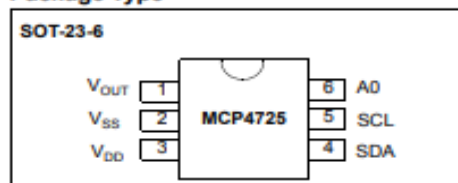


Рисунок Б.1 – Основные характеристики ЦАП МСР-4725

1.0 ELECTRICAL CHARACTERISTICS

Absolute Maximum Ratings†

V_{DD}	6.5V
All inputs and outputs w.r.t V_{SS}	-0.3V to $V_{DD}+0.3V$
Current at Input Pins	± 2 mA
Current at Supply Pins	± 50 mA
Current at Output Pins	± 25 mA
Storage Temperature	-65°C to +150°C
Ambient Temp. with Power Applied	-55°C to +125°C
ESD protection on all pins	≥ 6 kV HBM, $\geq 400V$ MM
Maximum Junction Temperature (T_J)	+150°C

† Notice: Stresses above those listed under "Maximum ratings" may cause permanent damage to the device. This is a stress rating only and functional operation of the device at these or any other conditions above those indicated in the operation listings of this specification is not implied. Exposure to maximum rating conditions for extended periods may affect device reliability.

ELECTRICAL CHARACTERISTICS

Electrical Specifications: Unless otherwise indicated, all parameters apply at $V_{DD} = +2.7V$ to $5.5V$, $V_{SS} = 0V$, $R_L = 5$ k Ω from V_{OUT} to V_{SS} , $C_L = 100$ pF, $T_A = -40^\circ C$ to $+125^\circ C$. Typical values are at $+25^\circ C$.

Parameter	Sym	Min	Typ	Max	Units	Conditions
Power Requirements						
Operating Voltage	V_{DD}	2.7		5.5	V	
Supply Current	I_{DD}	—	210	400	μA	Digital input pins are grounded, Output pin (V_{OUT}) is not connected (unloaded), Code = 000h
Power-Down Current	I_{DDP}	—	0.06	2.0	μA	$V_{DD} = 5.5V$
Power-On-Reset Threshold Voltage	V_{POR}	—	2	—	V	
DC Accuracy						
Resolution	n	12	—	—	Bits	Code Range = 000h to FFFh
INL Error	INL	—	± 2	± 14.5	LSB	Note 1
DNL	DNL	-0.75	± 0.2	± 0.75	LSB	Note 1
Offset Error	V_{OS}	—	0.02	0.75	% of FSR	Code = 000h
Offset Error Drift	$\Delta V_{OS}/^\circ C$	—	± 1	—	ppm/ $^\circ C$	-45°C to +25°C
		—	± 2	—	ppm/ $^\circ C$	+25°C to +85°C
Gain Error	G_E	-2	-0.1	2	% of FSR	Code = FFFh, Offset error is not included.
Gain Error Drift	$\Delta G_E/^\circ C$	—	-3	—	ppm/ $^\circ C$	
Output Amplifier						
Phase Margin	P_M	—	66	—	Degree(°)	$C_L = 400$ pF, $R_L = \infty$
Capacitive Load Stability	C_L	—	—	1000	pF	$R_L = 5$ k Ω , Note 2
Slew Rate	SR	—	0.55	—	V/ μs	
Short Circuit Current	I_{SC}	—	15	24	mA	$V_{DD} = 5V$, $V_{OUT} =$ Grounded
Output Voltage Settling Time	T_S	—	6	—	μs	Note 3

Note 1: Test Code Range: 100 to 4000.

2: This parameter is ensured by design and not 100% tested.

3: Within 1/2 LSB of the final value when code changes from 1/4 to 3/4 (400h to C00h) of full scale range.

4: Logic state of external address selection pin (A0 pin).

3.0 PIN DESCRIPTIONS

The descriptions of the pins are listed in Table 3-1.

TABLE 3-1: PIN FUNCTION TABLE

MCP4725	Name	Description
SOT-23		
1	V _{OUT}	Analog Output Voltage
2	V _{SS}	Ground Reference
3	V _{DD}	Supply Voltage
4	SDA	I ² C Serial Data
5	SCL	I ² C Serial Clock Input
6	A0	I ² C Address Bit Selection pin (A0 bit). This pin can be tied to V _{SS} or V _{DD} , or can be actively driven by the digital logic levels. The logic state of this pin determines what the A0 bit of the I ² C address bits should be.

3.1 Analog Output Voltage (V_{OUT})

V_{OUT} is an analog output voltage from the DAC device. DAC output amplifier drives this pin with a range of V_{SS} to V_{DD}.

3.2 Supply Voltage (V_{DD} or V_{SS})

V_{DD} is the power supply pin for the device. The voltage at the V_{DD} pin is used as the supply input as well as the DAC reference input. The power supply at the V_{DD} pin should be clean as possible for a good DAC performance.

This pin requires an appropriate bypass capacitor of about 0.1 μ F (ceramic) to ground. An additional 10 μ F capacitor (tantalum) in parallel is also recommended to further attenuate high frequency noise present in application boards. The supply voltage (V_{DD}) must be maintained in the 2.7V to 5.5V range for specified operation.

V_{SS} is the ground pin and the current return path of the device. The user must connect the V_{SS} pin to a ground plane through a low impedance connection. If an analog ground path is available in the application PCB (printed circuit board), it is highly recommended that the V_{SS} pin be tied to the analog ground path or isolated within an analog ground plane of the circuit board.

3.3 Serial Data Pin (SDA)

SDA is the serial data pin of the I²C interface. The SDA pin is used to write or read the DAC register and EEPROM data. The SDA pin is an open-drain N-channel driver. Therefore, it needs a pull-up resistor from the V_{DD} line to the SDA pin. Except for START and STOP conditions, the data on the SDA pin must be stable during the high period of the clock. The high or low state of the SDA pin can only change when the clock signal on the SCL pin is low. Refer to Section 7.0 "I²C Serial Interface Communication" for more details of I²C Serial Interface communication.

3.4 Serial Clock Pin (SCL)

SCL is the serial clock pin of the I²C interface. The MCP4725 acts only as a slave and the SCL pin accepts only external serial clocks. The input data from the Master device is shifted into the SDA pin on the rising edges of the SCL clock and output from the MCP4725 occurs at the falling edges of the SCL clock. The SCL pin is an open-drain N-channel driver. Therefore, it needs a pull-up resistor from the V_{DD} line to the SCL pin. Refer to Section 7.0 "I²C Serial Interface Communication" for more details of I²C Serial Interface communication.

3.5 Device Address Selection Pin (A0)

This pin is used to select the A0 address bit by the user. The user can tie this pin to V_{SS} (logic '0'), or V_{DD} (logic '1'), or can be actively driven by the digital logic levels, such as the I²C Master Output. See Section 7.2 "Device Addressing" for more details of the address bits.

MCP4725

TABLE 6-2: WRITE COMMAND TYPE

C2	C1	C0	Command Name	Function
0	0	X	Fast Mode	This command is used to change the DAC register. EEPROM is not affected
0	0	X	-	-
0	1	0	Write DAC Register	Load configuration bits and data code to the DAC Register
0	1	1	Write DAC Register and EEPROM	(a) Load configuration bits and data code to the DAC Register and (b) also write the EEPROM
1	0	0	Reserved	Reserved for future use
1	0	1	Reserved	Reserved for future use
1	1	0	Reserved	Reserved for future use
1	1	1	Reserved	Reserved for future use

Note 1: X = Don't Care. Fast Mode does not use C0 bit.

2: The MCP4725 ignores the "Reserved" commands.

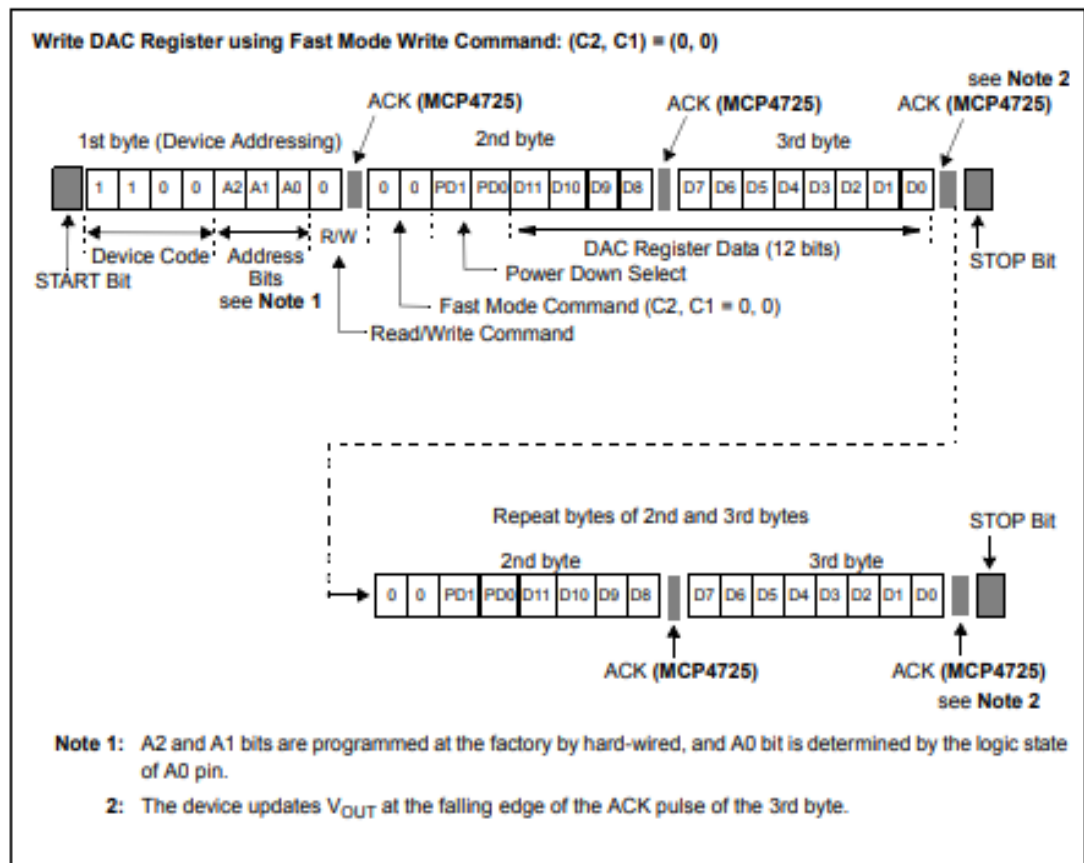


FIGURE 6-1: Fast Mode Write Command.

Техническая документация микроконтроллерной платы Arduino Uno представлена на
Рисунок Б.5–6



Общие сведения

Arduino Uno - это устройство на основе микроконтроллера ATmega328 ([datasheet](#)). В его состав входит все необходимое для удобной работы с микроконтроллером: 14 цифровых входов/выходов (из них 6 могут использоваться в качестве ШИМ-выходов), 6 аналоговых входов, кварцевый резонатор на 16 МГц, разъем USB, разъем питания, разъем для внутрисхемного программирования (ICSP) и кнопка сброса. Для начала работы с устройством достаточно просто подать питание от AC/DC-адаптера или батарейки, либо подключить его к компьютеру посредством USB-кабеля.

В отличие от всех предыдущих плат Ардуино, Uno в качестве преобразователя интерфейсов USB-UART использует микроконтроллер ATmega16U2 (ATmega8U2 до версии R2) вместо микросхемы FTDI.

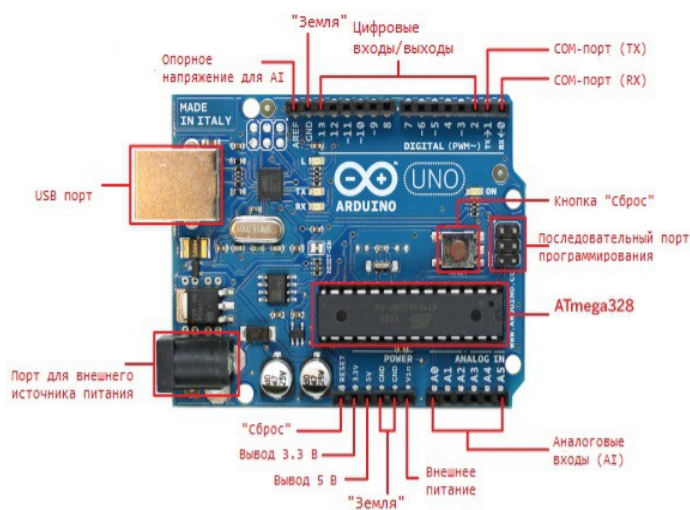
На плате Arduino Uno версии R2 для упрощения процесса обновления прошивки добавлен резистор, подтягивающий к земле линию HWB микроконтроллера 8U2.

Изменения на плате версии R3 перечислены ниже:

- Распиновка 1.0: добавлены выводы SDA и SCL (возле вывода AREF), а также два новых вывода, расположенных возле вывода RESET. Первый - IOREF - позволяет платам расширения подстраиваться под рабочее напряжение Ардуино. Данный вывод предусмотрен для совместимости плат расширения как с 5В-Ардуино на базе микроконтроллеров AVR, так и с 3.3В-платами Arduino Due. Второй вывод ни к чему не подсоединен и зарезервирован для будущих целей.
- Улучшена помехоустойчивость цепи сброса.
- Микроконтроллер ATmega8U2 заменен на ATmega16U2.

"Uno" (в переводе с итальянского - "один") назван по случаю предстоящего выпуска Arduino 1.0. Совместно с Arduino 1.0 данные устройства будут базовыми версиями Ардуино. Uno - эталонная модель платформы Arduino и является последней в серии USB-плат;

Рисунок Б.5 – Общие сведения о микроконтроллерной плате Arduino Uno R3



Характеристики

Микроконтроллер	ATmega328
Рабочее напряжение	5В
Напряжение питания (рекомендуемое)	7-12В
Напряжение питания (предельное)	6-20В
Цифровые входы/выходы	14 (из них 6 могут использоваться в качестве ШИМ-выходов)
Аналоговые входы	6
Максимальный ток одного вывода	40 мА
Максимальный выходной ток вывода 3.3В	50 мА
Flash-память	32 КБ (ATmega328) из которых 0.5 КБ используются загрузчиком
SRAM	2 КБ (ATmega328)
EEPROM	1 КБ (ATmega328)
Тактовая частота	16 МГц

Рисунок Б.6 – Общие сведения о микроконтроллерной плате Arduino Uno

В Таблице Б.1 представлены технические характеристики осциллографа DSO138.

Таблица Б.1 – Технические характеристики осциллографа DSO138

Параметр	Значение
Макс. частота дискрет.	1 Мвыб/с
Аналоговый диапазон	0...200 КГц
Чувствительность	10 мВ/дел ... 5 В/дел
Макс. вход. напряжение	50 V _{pk} (щуп 1X)
Вход. сопротивление	1 МОм / 20 пФ
Разрешение	12 бит
Длина данных	1024 точки
Временной диапазон	500 с/дел...10 мкс/дел
Режимы триггера	Авто, норм., однократ.
Диапазон полож. триг.	50%
Питание	9В DC (8...12В)
Потребление тока	~120 мА
Размеры	117 x 76 x 15 мм
Масса	70 грамм (без щупа)

ПРИЛОЖЕНИЕ В

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <math.h>

#define PI 3.141592653589793

int okrugli(double x) {
    if (x >= 0) {
        return (int)(x + 0.5);
    }
    else {
        return (int)(x - 0.5);
    }
}

int otbros(double x) {
    return (int)x;
}

int znak(double x) {
    if (x > 0) return 1;
    if (x < 0) return -1;
    return 0;
}

double pervaya_garmonika_int(int n, int* signal, double k) {
    double sum_sin = 0.0;
    double sum_cos = 0.0;
    int i;
    double ugol;

    for (i = 0; i <= n; i++) {
        ugol = (2.0 * i + 1.0) * PI / n;
        sum_sin = sum_sin + (double)signal[i] * sin(ugol);
        sum_cos = sum_cos + (double)signal[i] * cos(ugol);
    }

    return sqrt(sum_sin * sum_sin + sum_cos * sum_cos) * k;
}

double pervaya_garmonika_double(int n, double* signal, double k) {
    double sum_sin = 0.0;
    double sum_cos = 0.0;
    int i;
    double ugol;

    for (i = 0; i <= n; i++) {
        ugol = (2.0 * i + 1.0) * PI / n;
        sum_sin = sum_sin + signal[i] * sin(ugol);
    }
}
```

```

        sum_cos = sum_cos + signal[i] * cos(ugol);
    }

    return sqrt(sum_sin * sum_sin + sum_cos * sum_cos) * k;
}

double oshibka_sksz(int n, double* ideal, int* quant) {
    double sum_kv_ideal = 0.0;
    double sum_kv_quant = 0.0;
    double rms_ideal, rms_quant;
    int i;

    for (i = 0; i <= n; i++) {
        sum_kv_ideal = sum_kv_ideal + (ideal[i] * ideal[i]);
        sum_kv_quant = sum_kv_quant + ((double)quant[i] * (double)quant[i]);
    }

    rms_ideal = sqrt(sum_kv_ideal / (n + 1));
    rms_quant = sqrt(sum_kv_quant / (n + 1));

    return (fabs(rms_ideal - rms_quant) / rms_ideal) * 100.0;
}

int main() {
    int n_values[] = { 2, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100 };
    int n_count = 11;

    double a_values[] = { 200.0, 500.0, 1000.0, 1500.0, 2000.0 };
    int a_count = 5;

    double ideal[101];
    int vniz[101];
    int vverh[101];
    int bliz[101];
    int adapt[101];

    int n, i, j;
    int idx_n, idx_a;

    double ugol;
    double k;
    double garm_ideal;

    double sum_sin_real, sum_cos_real;
    double base_sin, base_cos;
    double u, sin_u, cos_u;
    double garm_tek, garm_verh, garm_niz;
    double oshibka_tek, oshibka_verh, oshibka_niz;
    int old_val, x_bliz, x_vverh, x_vniz, best_val;

    double garm_vniz, garm_vverh, garm_bliz, garm_adapt;
    double osh_garm_vniz, osh_garm_vverh, osh_garm_bliz, osh_garm_adapt;

```

```

double osh_skz_vniz, osh_skz_vverh, osh_skz_bliz, osh_skz_adapt;
double snizhenie;

FILE* f = fopen("rezultat.txt", "w");
if (f == NULL) {
    printf("Oshibka: ne mogu sozdat fail rezultat.txt\n");
    return 1;
}

fprintf(f, "N\tA\tMetod\tPogr_1_garm(%%)\tPogr_SKZ(%%)\tSnizhenie(raz)\n");

for (idx_n = 0; idx_n < n_count; idx_n++) {
    n = n_values[idx_n];

    for (idx_a = 0; idx_a < a_count; idx_a++) {
        double A = a_values[idx_a];

        printf("Schitaju dlja N = %d, A = %.0f ... \n", n, A);

        for (i = 0; i <= n; i++) {
            ugol = 2.0 * PI * i / (1.001 * n) - PI / n;
            ideal[i] = A * sin(ugol);
        }

        for (i = 0; i <= n; i++) {
            vniz[i] = otbros(ideal[i]);
            vverh[i] = otbros(ideal[i]) + znak(ideal[i]);
            bliz[i] = okrugli(ideal[i]);
            adapt[i] = bliz[i];
        }

        k = (2.0 / n) * sin(PI / n) / (PI / n);

        garm_ideal = pervaya_garmonika_double(n, ideal, k);

        sum_sin_real = 0.0;
        sum_cos_real = 0.0;
        for (j = 0; j <= n; j++) {
            ugol = (2.0 * j + 1.0) * PI / n;
            sum_sin_real = sum_sin_real + (double)adapt[j] * sin(ugol);
            sum_cos_real = sum_cos_real + (double)adapt[j] * cos(ugol);
        }

        for (i = 0; i <= n; i++) {
            old_val = adapt[i];
            u = (2.0 * i + 1.0) * PI / n;
            sin_u = sin(u);
            cos_u = cos(u);

            base_sin = sum_sin_real - (double)old_val * sin_u;
            base_cos = sum_cos_real - (double)old_val * cos_u;
        }
    }
}

```

```

x_bliz = okrugli(ideal[i]);
x_vniz = otbros(ideal[i]);
x_vverh = otbros(ideal[i]) + znak(ideal[i]);

garm_tek = sqrt((base_sin + (double)x_bliz * sin_u) * (base_sin + (double)x_bliz * sin_u)
+
(base_cos + (double)x_bliz * cos_u) * (base_cos + (double)x_bliz * cos_u)) * k;
oshibka_tek = fabs(garm_tek - garm_ideal);

garm_verh = sqrt((base_sin + (double)x_vverh * sin_u) * (base_sin + (double)x_vverh *
sin_u) +
(base_cos + (double)x_vverh * cos_u) * (base_cos + (double)x_vverh * cos_u)) * k;
oshibka_verh = fabs(garm_verh - garm_ideal);

garm_niz = sqrt((base_sin + (double)x_vniz * sin_u) * (base_sin + (double)x_vniz *
sin_u) +
(base_cos + (double)x_vniz * cos_u) * (base_cos + (double)x_vniz * cos_u)) * k;
oshibka_niz = fabs(garm_niz - garm_ideal);

best_val = x_bliz;
if (oshibka_verh < oshibka_tek && oshibka_verh < oshibka_niz) {
    best_val = x_vverh;
}
if (oshibka_niz < oshibka_tek && oshibka_niz < oshibka_verh) {
    best_val = x_vniz;
}

if (best_val != old_val) {
    adapt[i] = best_val;
    sum_sin_real = base_sin + (double)best_val * sin_u;
    sum_cos_real = base_cos + (double)best_val * cos_u;
}
}

garm_vniz = pervaya_garmonika_int(n, vniz, k);
garm_vverh = pervaya_garmonika_int(n, vverh, k);
garm_bliz = pervaya_garmonika_int(n, bliz, k);
garm_adapt = pervaya_garmonika_int(n, adapt, k);

osh_garm_vniz = fabs(garm_vniz - garm_ideal) / garm_ideal * 100.0;
osh_garm_vverh = fabs(garm_vverh - garm_ideal) / garm_ideal * 100.0;
osh_garm_bliz = fabs(garm_bliz - garm_ideal) / garm_ideal * 100.0;
osh_garm_adapt = fabs(garm_adapt - garm_ideal) / garm_ideal * 100.0;

osh_skz_vniz = oshibka_skz(n, ideal, vniz);
osh_skz_vverh = oshibka_skz(n, ideal, vverh);
osh_skz_bliz = oshibka_skz(n, ideal, bliz);
osh_skz_adapt = oshibka_skz(n, ideal, adapt);

fprintf(f, "%d\t%.0f\tvniz\t%.6f\t%.6f\t---\n",
n, A, osh_garm_vniz, osh_skz_vniz);
fprintf(f, "%d\t%.0f\tvverh\t%.6f\t%.6f\t---\n",

```

```

        n, A, osh_garm_vverh, osh_skz_vverh);
    fprintf(f, "%d\t%.0f\tbliz\t%.6f\t%.6f\t---\n",
        n, A, osh_garm_bliz, osh_skz_bliz);

    if (osh_garm_adapt < 0.000001) {
        fprintf(f, "%d\t%.0f\tadapt\t%.6f\t%.6f\t>1000\n",
            n, A, osh_garm_adapt, osh_skz_adapt);
    }
    else {
        snizhenie = osh_garm_bliz / osh_garm_adapt;
        fprintf(f, "%d\t%.0f\tadapt\t%.6f\t%.6f\t%.1f\n",
            n, A, osh_garm_adapt, osh_skz_adapt, snizhenie);
    }
}
}

fclose(f);

return 0;
}

```

ПРИЛОЖЕНИЕ Г

На Рисунок Г.1 представлен снимок экрана, демонстрирующий первые строчки файла, созданного программой после на языке программирования С в среде разработки Visual Studio.

N	A	Metod	Pogr_1_garm(%)	Pogr_SKZ(%)	Snizhenie(raz)
2	200	vniz	0.332515	0.332237	---
2	200	vverh	0.167489	0.167766	---
2	200	bliz	0.000821	0.000821	---
2	200	adapt	0.000821	0.000821	1.0
2	500	vniz	0.132514	0.132469	---
2	500	vverh	0.067488	0.067532	---
2	500	bliz	0.000821	0.000821	---
2	500	adapt	0.000821	0.000821	1.0
2	1000	vniz	0.065846	0.065835	---
2	1000	vverh	0.034154	0.034166	---
2	1000	bliz	0.000821	0.000821	---
2	1000	adapt	0.000821	0.000821	1.0
2	1500	vniz	0.043624	0.043619	---
2	1500	vverh	0.023043	0.023048	---
2	1500	bliz	0.000821	0.000821	---
2	1500	adapt	0.000821	0.000821	1.0
2	2000	vniz	0.032513	0.032510	---
2	2000	vverh	0.017488	0.017490	---
2	2000	bliz	0.000821	0.000821	---
2	2000	adapt	0.000821	0.000821	1.0
10	200	vniz	0.410455	0.405269	---
10	200	vverh	0.257780	0.260056	---
10	200	bliz	0.043243	0.038829	---
10	200	adapt	0.003482	0.010190	12.4
10	500	vniz	0.125413	0.125543	---
10	500	vverh	0.141897	0.140629	---
10	500	bliz	0.023088	0.023052	---
10	500	adapt	0.001067	0.001339	21.6
10	1000	vniz	0.074251	0.074323	---
10	1000	vverh	0.059403	0.058753	---
10	1000	bliz	0.027992	0.028945	---
10	1000	adapt	0.000806	0.000975	34.7
10	1500	vniz	0.057197	0.057169	---
10	1500	vverh	0.031906	0.031547	---
10	1500	bliz	0.002008	0.001824	---
10	1500	adapt	0.001263	0.002171	1.6
10	2000	vniz	0.051121	0.051639	---
10	2000	vverh	0.015706	0.014894	---
10	2000	bliz	0.008532	0.008803	---
10	2000	adapt	0.001474	0.002141	5.8
20	200	vniz	0.334651	0.333590	---
20	200	vverh	0.311195	0.311869	---
20	200	bliz	0.007088	0.006789	---

Рисунок Г.1 – Первые строчки файла с данными об относительной погрешности первой гармоник и СКЗ при разной амплитуде и количестве отсчётов на период